

SLOT 10: Authentication and Security (Cont)

Tóm tắt

28 User Authentication With Passport

- Use Passport Module Together With Passport-Local

29.1 use passport-local-mongoose for setting up local authentication within server

29.2 use passport together with sessions to set up user authentication

30 Token Based Authentication



Authentication and Security



28. User Authentication With Passport

Tổng quan: Sử dụng thư viện Passport để thực hiện xác thực người dùng trong ứng dụng Node.js.

- **Khái niệm chính:**
 - Passport là middleware xác thực linh hoạt cho Node.js.
 - Hỗ trợ nhiều chiến lược xác thực như: Local, OAuth (Google, Facebook), JWT,...
- **Mục tiêu bài học:**
 - Hiểu cách tích hợp Passport vào ứng dụng Express.
 - Biết cách sử dụng Passport để xác thực người dùng dựa trên tài khoản local (tài khoản do ứng dụng quản lý).



29.1 Sử dụng passport-local-mongoose để thiết lập xác thực local

Tổng quan: Cấu hình xác thực người dùng bằng tên đăng nhập và mật khẩu với Mongoose.

- **Khái niệm chính:**
 - `passport-local`: Chiến lược xác thực dựa trên username/password.
 - `passport-local-mongoose`: Plugin hỗ trợ tự động thêm trường username, password và phương thức xác thực vào schema Mongoose.
- **Mục tiêu bài học:**
 - Tạo schema người dùng với Mongoose và tích hợp `passport-local-mongoose`.
 - Thiết lập xác thực local đơn giản, bảo mật.



29.2 Kết hợp Passport với phiên (Sessions) để xác thực người dùng

Tổng quan: Duy trì trạng thái đăng nhập của người dùng thông qua session.

- **Khái niệm chính:**
 - Session là cơ chế lưu trữ thông tin người dùng giữa các request.
 - `express-session`: Middleware để quản lý session trong Express.
 - Passport sử dụng session để nhớ trạng thái đăng nhập của người dùng.
- **Mục tiêu bài học:**
 - Cấu hình session và tích hợp với Passport.
 - Hiểu cách Passport serialize/deserialize user để duy trì phiên đăng nhập.



30. Token Based Authentication (Xác thực dựa trên Token)

Tổng quan: Sử dụng token thay vì session để xác thực người dùng – thường dùng trong API RESTful và ứng dụng đa nền tảng.

- **Khái niệm chính:**
 - Token (thường là JWT – JSON Web Token): chuỗi mã hóa chứa thông tin người dùng.
 - Không cần lưu trữ session server-side → stateless.

- Client gửi token trong header mỗi request để chứng minh danh tính.
- **Mục tiêu bài học:**
 - Hiểu mô hình xác thực không dùng session (token-based).
 - Biết cách tạo, verify và sử dụng JWT trong ứng dụng Node.js.

Ghi chú tổng quan:

- Các bài học này hướng đến việc xây dựng hệ thống xác thực đầy đủ và an toàn cho ứng dụng web/API.
- Học viên sẽ hiểu cả hai mô hình xác thực phổ biến: **session-based** và **token-based**, từ đó chọn lựa phù hợp với từng loại ứng dụng cụ thể (web app truyền thống vs API backend cho mobile/app).

TÀI LIỆU THAM KHẢO

1. "Node.js Design Patterns" – Mario Casciaro

- **Mô tả:** Cuốn sách kinh điển về các mẫu thiết kế và kiến trúc ứng dụng Node.js chuyên nghiệp, bao gồm phần chi tiết về bảo mật, middleware, session, và xác thực.
- **Chương liên quan:**
 - Chương về "Security in Node.js"
 - Chương về "Passport.js và quản lý người dùng"
- **Phù hợp cho:** Sinh viên muốn hiểu bản chất hoạt động của Node.js từ bên trong, kết hợp với thực hành nâng cao.

 <https://www.packtpub.com/product/node-js-design-patterns-third-edition/9781803239363>

2. Tài liệu chính thức của Passport.js

- **Mô tả:** Tài liệu gốc của thư viện Passport.js, cung cấp đầy đủ hướng dẫn sử dụng các chiến lược xác thực khác nhau như `passport-local`, `passport-jwt`, `OAuth`,...
- **Điểm mạnh:**
 - Cập nhật liên tục
 - Ví dụ cụ thể cho từng strategy
- **Link:**

 <http://www.passportjs.org/docs/>

3. FreeCodeCamp – JSON Web Token and Authentication Tutorial

- **Mô tả:** Video hướng dẫn miễn phí trên YouTube bởi kênh FreeCodeCamp, giải thích rõ ràng cách hoạt động của JWT và tích hợp vào ứng dụng Node.js.
- **Thời lượng:** ~1 giờ
- **Nội dung nổi bật:**
 - Cách tạo và verify token
 - Xác thực không dùng session
 - Bảo mật API với JWT

 <https://www.youtube.com/watch?v=mbsmsi7IPI4>

4. Auth0 – Learn Identity & Access Management

- **Mô tả:** Trang học tập và hướng dẫn của Auth0 – một trong những nền tảng xác thực phổ biến nhất hiện nay.
- **Các bài viết nên đọc:**
 - "What is JWT?"
 - "Introduction to OAuth 2.0 and OpenID Connect"
 - "How to Secure Node.js APIs with JWT"
- **Link:**

 <https://auth0.com/learn/>

5. Udemy – “Node.js Advanced Authentication with Passport, JWT, OAuth2”

- **Mô tả:** Khóa học trên Udemy đi sâu vào các phương pháp xác thực nâng cao như Passport, JWT, Google/Facebook OAuth, Refresh Token, v.v.
- **Ưu điểm:**
 - Thực hành theo dự án cụ thể
 - Có demo full-stack (Frontend + Backend)
- **Link:**
 <https://www.udemy.com/course/advanced-node-for-developers/>

CÁC NỘI DUNG TRỌNG TÂM

Chủ đề 1: Xác thực với Passport.js

Mục tiêu: Sinh viên hiểu cách tích hợp xác thực người dùng vào ứng dụng web Node.js bằng thư viện Passport.

Khái niệm quan trọng

- Passport.js: middleware xác thực linh hoạt cho Node.js.
- Strategy (Chiến lược): cơ chế xác thực như Local, OAuth2, JWT,...
- Middleware: cách Passport tích hợp vào Express để xử lý đăng nhập.

Kỹ thuật cần thành thạo

- Cài đặt và cấu hình `passport`, `passport-local`, `passport-local-mongoose`.
- Tích hợp với Express thông qua middleware `passport.initialize()` và `passport.session()`.
- Viết hàm xác thực người dùng (custom verify callback).

Lỗi thường gặp

- Không gọi đúng thứ tự các middleware của Passport → không lưu session.
- Thiếu `serializeUser` / `deserializeUser` → không duy trì trạng thái đăng nhập.
- Lỗi logic khi kiểm tra username/password trong verify function.

Best Practices

- Luôn sử dụng `express-session` khi làm việc với Passport sessions.
- Phân tách cấu hình Passport ra file riêng để dễ bảo trì.
- Bảo vệ route bằng middleware kiểm tra `req.isAuthenticated()`.

Chủ đề 2: Phiên đăng nhập (Session-based Authentication)

Mục tiêu: Sinh viên hiểu và áp dụng mô hình xác thực dựa trên phiên – phù hợp cho ứng dụng web truyền thống.

Khái niệm quan trọng

- Session: dữ liệu lưu trữ tạm thời giữa server và client.
- Cookie: chứa session ID gửi từ client lên server mỗi lần request.
- `express-session`: module phổ biến để quản lý session trong Express.

Kỹ thuật cần thành thạo

- Cấu hình `express-session` với store phù hợp (`MemoryStore`, `MongoDBStore`,...).
- Sử dụng `passport.serializeUser()` và `passport.deserializeUser()`.
- Kiểm soát trạng thái đăng nhập và chuyển hướng người dùng.

Lỗi thường gặp

- Không cấu hình secret key cho session → rủi ro bảo mật.
- Dùng `MemoryStore` trong môi trường production → mất dữ liệu khi restart server.
- Không xóa session sau khi logout → bảo mật kém.

✓ Best Practices

- Sử dụng session store bền vững như Redis hoặc MongoDB trong production.
- Đặt thời gian sống cho session (`maxAge`) hợp lý.
- Xóa session hoàn toàn khi người dùng logout.

🔑 Chủ đề 3: Xác thực dựa trên Token (Token-based Authentication)

Mục tiêu: Sinh viên hiểu và triển khai xác thực không dùng session bằng token, đặc biệt là JWT.

✓ Khái niệm quan trọng

- JSON Web Token (JWT): định dạng token mã hóa chứa payload người dùng.
- Stateless authentication: không lưu trữ trạng thái trên server.
- Header, Payload, Signature: 3 phần cấu thành của JWT.

✂ Kỹ thuật cần thành thạo

- Tạo JWT bằng `jsonwebtoken` (`jwt.sign()`).
- Verify token từ header Authorization (`jwt.verify()`).
- Tích hợp JWT vào API route bảo vệ.

⚠ Lỗi thường gặp

- Gửi token qua HTTP không bảo mật → bị đánh cắp.
- Không kiểm tra hết hạn của token (`exp` claim).
- Lưu trữ token không an toàn ở phía client.

✓ Best Practices

- Luôn sử dụng HTTPS khi truyền token.
- Chỉ cấp token có thời hạn ngắn, dùng refresh token để gia hạn.
- Lưu trữ token ở phía client bằng `HttpOnly` cookie hoặc `localStorage` tùy ngữ cảnh.

📦 Tổng kết kỹ năng cần đạt được sau bài học

Sau bài học này, sinh viên phải:

- Biết cách chọn mô hình xác thực phù hợp: session-based (cho web app) hay token-based (cho API, mobile app).
- Thành thạo tích hợp Passport.js với nhiều chiến lược khác nhau.
- Có thể xây dựng hệ thống đăng nhập đầy đủ: register, login, logout, protect route.
- Hiểu và tránh được các lỗi phổ biến liên quan đến bảo mật xác thực như: session fixation, token leakage, insecure cookies,...

ỨNG DỤNG THỰC TIỄN

PHẦN 1: 3 TÌNH HUỐNG THỰC TẾ

Tình huống 1: Ứng dụng Thương mại điện tử (E-commerce)

Mô tả bối cảnh:

Một startup xây dựng nền tảng mua bán trực tuyến, nơi người dùng có thể đăng ký tài khoản, mua hàng, xem lịch sử đơn hàng và thanh toán online.

Cách áp dụng kỹ thuật:

- **Passport.js + Local Strategy** : Xác thực người dùng bằng email/mật khẩu.
- **Session-based authentication** : Giữ trạng thái đăng nhập khi người dùng duyệt web.
- **JWT** : Cho phép truy cập API từ mobile app với token thay vì session.
- **bcrypt / passport-local-mongoose** : Mã hóa mật khẩu trước khi lưu trữ.

Mục tiêu:

- Bảo vệ thông tin cá nhân và giao dịch của người dùng.
- Ngăn chặn truy cập trái phép vào tài khoản.
- Đảm bảo tính nhất quán giữa Web App và Mobile App.

Tình huống 2: Hệ thống Y tế điện tử (e-Healthcare)

Mô tả bối cảnh:

Một bệnh viện triển khai hệ thống quản lý hồ sơ sức khỏe điện tử. Bệnh nhân và bác sĩ đều có tài khoản để truy cập thông tin nhạy cảm như kết quả xét nghiệm, tiền sử điều trị,...

Cách áp dụng kỹ thuật:

- **Token-based Authentication (JWT)** : Tránh lưu trữ session trên server nhằm đảm bảo tính stateless cho hệ thống phân tán.
- **Role-based Access Control (RBAC)** : Chỉ định quyền truy cập dựa trên vai trò (bệnh nhân, bác sĩ, admin).
- **HttpOnly Cookie + Refresh Token** : Lưu trữ token an toàn và gia hạn phiên đăng nhập.

Mục tiêu:

- Bảo vệ dữ liệu y tế nhạy cảm.
- Phân quyền chính xác để tránh truy cập không mong muốn.
- Đảm bảo tuân thủ các quy định pháp lý về bảo mật thông tin y tế.

Tình huống 3: Ứng dụng Ngân hàng số (Digital Banking App)

Mô tả bối cảnh:

Một ngân hàng phát triển ứng dụng di động giúp khách hàng tra cứu số dư, chuyển tiền, thanh toán hóa đơn và xem lịch sử giao dịch.

Cách áp dụng kỹ thuật:

- **JWT + Refresh Token** : Cấp phép truy cập API an toàn, có thời hạn.
- **HTTPS bắt buộc** : Bảo vệ dữ liệu truyền đi khỏi bị nghe lén.
- **Logout và hủy token** : Sử dụng Redis hoặc Blacklist để vô hiệu hóa token sau khi đăng xuất.
- **Rate limiting** : Ngăn chặn tấn công brute-force.

Mục tiêu:

- Ngăn chặn đánh cắp thông tin tài chính.
- Đảm bảo mỗi giao dịch đều được xác thực rõ ràng.
- Tăng độ tin cậy và trải nghiệm người dùng qua cơ chế xác thực đa lớp.

PHẦN 2: CASE STUDY – “HealthConnect”: Ứng dụng chăm sóc sức khỏe cộng đồng

Giới thiệu dự án:

HealthConnect là một startup cung cấp nền tảng kết nối bệnh nhân với bác sĩ tư vấn trực tuyến. Sau một thời gian vận hành, họ gặp phải nhiều vấn đề liên quan đến bảo mật tài khoản người dùng và rò rỉ thông tin y tế.

! Vấn đề bảo mật gặp phải:

- Người dùng phàn nàn rằng tài khoản bị chiếm đoạt.
- Hacker có thể đoán mật khẩu dễ dàng do mật khẩu chưa được mã hóa đúng cách.
- Phiên đăng nhập không hết hạn dẫn đến tình trạng "session hijacking".

✅ Giải pháp áp dụng:

- Sử dụng `passport-local-mongoose` để tự động hash mật khẩu trước khi lưu vào MongoDB.
- Kết hợp `Passport.js` với `Express-session` để duy trì phiên đăng nhập an toàn.
- Thiết lập JWT cho API backend phục vụ ứng dụng di động.
- Thêm middleware kiểm tra role người dùng để phân quyền truy cập.
- Triển khai logout hoàn chỉnh, xóa session và loại bỏ token khỏi blacklist.

🧠 Bài học rút ra:

- Không bao giờ lưu mật khẩu dưới dạng plain text – luôn mã hóa bằng bcrypt hoặc tương tự.
- Phiên đăng nhập cần có thời gian sống giới hạn, và phải có khả năng hủy bỏ khi cần thiết.
- Phân quyền rõ ràng và kiểm soát chặt chẽ API endpoint là yếu tố then chốt để ngăn chặn truy cập trái phép.
- Xác thực đa tầng (multi-factor) nên được cân nhắc trong các ứng dụng xử lý dữ liệu nhạy cảm.

🔗 Kết luận – Kết nối lý thuyết với thực tiễn

Thông qua các tình huống và case study trên, sinh viên sẽ hiểu rõ hơn về:

- Vai trò thực tế của xác thực và bảo mật trong từng loại hình ứng dụng.
- Cách lựa chọn mô hình phù hợp: session hay token?
- Các kỹ thuật nào cần áp dụng để đảm bảo dữ liệu không bị rò rỉ hay giả mạo.
- Bài học từ các sai lầm thường gặp trong quá trình phát triển sản phẩm thực tế.

CHI TIẾT MỘT SỐ NỘI DUNG



Kỹ thuật 28: User Authentication With Passport- Dùng Passport + Passport-Local



Mô tả

Passport.js là thư viện middleware xác thực linh hoạt cho ứng dụng Node.js, hỗ trợ nhiều chiến lược xác thực khác nhau (local, OAuth, JWT...). Trong đó, `passport-local` là chiến lược dùng để xác thực người dùng bằng username và password – rất phổ biến trong các ứng dụng web truyền thống.



Khái niệm & Vai trò

- **Passport** : cung cấp giao diện trung lập để tích hợp nhiều phương thức xác thực.
- **Passport-Local** : chiến lược xác thực dựa trên tài khoản local (do chính hệ thống quản lý).
- **Vai trò trong bảo mật** :
 - Kiểm soát việc đăng nhập người dùng.
 - Phân biệt giữa người dùng hợp lệ và không hợp lệ.
 - Là bước đầu tiên trong quy trình bảo vệ route hoặc API endpoint.



Các bước triển khai

Bước 1: Cài đặt các thư viện cần thiết

bash

```
npm install express passport passport-local
```

Bước 2: Khởi tạo Passport trong Express

js

```
const express = require('express');
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
```

```
const app = express();
```

```
// Middleware để parse body
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

```
// Khởi tạo Passport
app.use(passport.initialize());
```

Bước 3: Cấu hình Local Strategy

js

```
passport.use(new LocalStrategy(
  function(username, password, done) {
    // Giả sử có hàm tìm user trong DB
    User.findOne({ username: username }, function(err, user) {
      if (err) return done(err);
      if (!user) return done(null, false, { message: 'Tài khoản không tồn tại' });
      if (!user.verifyPassword(password)) return done(null, false, { message: 'Mật khẩu sai' });
    });
    return done(null, user);
  });
```

Bước 4: Tạo route login

js

```
app.post('/login',
  passport.authenticate('local', { failureRedirect: '/login' }),
  function(req, res) {
    res.redirect('/profile');
```

```
}  
);
```

Ví dụ đầy đủ đơn giản (User Model giả lập)

js

```
// models/User.js
```

```
const users = [  
  { id: 1, username: 'admin', password: '123456' } // Nên hash password trong môi trường thật  
];
```

```
module.exports = {  
  findOne: (query, callback) => {  
    const user = users.find(u => u.username === query.username);  
    callback(null, user);  
  }  
};
```

js

```
// app.js
```

```
const express = require('express');  
const passport = require('passport');  
const LocalStrategy = require('passport-local').Strategy;  
const User = require('./models/User');
```

```
const app = express();
```

```
app.use(express.urlencoded({ extended: false }));  
app.use(passport.initialize());
```

```
passport.use(new LocalStrategy(  
  function(username, password, done) {  
    User.findOne({ username: username }, function(err, user) {  
      if (err) return done(err);  
      if (!user) return done(null, false, { message: 'Không tìm thấy tài khoản' });  
      if (user.password !== password) return done(null, false, { message: 'Sai mật khẩu' });  
      return done(null, user);  
    });  
  }  
));
```

```
app.post('/login',  
  passport.authenticate('local', { failureRedirect: '/login' }),  
  function(req, res) {  
    res.send('Đăng nhập thành công!');  
  });
```

```
app.listen(3000, () => console.log('Server chạy ở port 3000'));
```

Lỗi thường gặp

Không đăng nhập được dù đúng tài khoản	Thiếu <code>done()</code> hoặc logic kiểm tra sai	Kiểm tra logic xác thực password
Không gọi <code>passport.initialize()</code>	Passport chưa được khởi tạo	Thêm middleware
Route <code>/login</code> không xử lý khi thất bại	Chưa redirect hoặc trả lỗi	Sử dụng option <code>{ failureRedirect }</code> hoặc custom handler

Best Practices

- Luôn hash mật khẩu trước khi lưu vào DB → xem phần `passport-local-mongoose`.
- Dùng HTTPS để bảo vệ dữ liệu đăng nhập.
- Hạn chế thông báo lỗi quá chi tiết (tránh leak thông tin).

- Kết hợp với session để duy trì trạng thái đăng nhập.

Liên kết với các kỹ thuật khác

- **Kết hợp với passport-local-mongoose** (xem phần tiếp theo): tự động mã hóa mật khẩu.
- **Kết hợp với session** (phần 29.2): duy trì trạng thái đăng nhập.
- **Kết hợp với JWT** (phần 30): chuyển sang xác thực token cho mobile/API.

Kỹ thuật 29.1: Sử dụng passport-local-mongoose để thiết lập xác thực local

Mục tiêu bài học:

Giúp sinh viên hiểu cách tích hợp passport-local-mongoose vào ứng dụng Node.js để xây dựng hệ thống xác thực local (username/password) một cách an toàn và hiệu quả.

Khái niệm & Vai trò

passport-local-mongoose là gì?

passport-local-mongoose là một plugin cho **Mongoose** giúp tự động thêm các phương thức và trường cần thiết để hỗ trợ xác thực người dùng bằng username và password.

Nó tích hợp liền mạch với **Passport.js + Local Strategy**, đồng thời xử lý việc:

- Tự động mã hóa mật khẩu (hash)
- Cung cấp phương thức kiểm tra mật khẩu (comparePassword)
- Hỗ trợ đăng ký người dùng dễ dàng

Vai trò trong bảo mật:

- Ngăn chặn lưu trữ mật khẩu thô → giảm rủi ro khi DB bị xâm nhập.
- Đơn giản hóa quy trình xác thực người dùng.
- Là nền tảng để xây dựng hệ thống đăng nhập hoàn chỉnh với Passport.js.

Các bước triển khai cụ thể trong Node.js

Bước 1: Cài đặt thư viện cần thiết

bash

```
npm install express mongoose passport passport-local passport-local-mongoose
```

Bước 2: Tạo User Schema với passport-local-mongoose

js

```
// models/User.js
const mongoose = require('mongoose');
const passportLocalMongoose = require('passport-local-mongoose');

const UserSchema = new mongoose.Schema({
  // Có thể thêm các field khác như email, fullname, ...
});

// Plugin passport-local-mongoose sẽ tự động thêm username, hash password, ...
UserSchema.plugin(passportLocalMongoose);

module.exports = mongoose.model('User', UserSchema);
```

Bước 3: Khởi tạo Passport và cấu hình Local Strategy

js

```
// config/passport.js
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const User = require('../models/User');
```

```
passport.use(new LocalStrategy(User.authenticate()));
passport.serializeUser(User.serializeUser());
passport.deserializeUser(User.deserializeUser());
```

Lưu ý: `User.authenticate()`, `User.serializeUser()` và `User.deserializeUser()` đều được cung cấp bởi `passport-local-mongoose`.

Bước 4: Cấu hình Express App

js

```
// app.js
const express = require('express');
const mongoose = require('mongoose');
const session = require('express-session');
const passport = require('passport');
const User = require('./models/User');

const app = express();

// Kết nối MongoDB
mongoose.connect('mongodb://localhost:27017/authDemo', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

// Middleware
app.use(express.urlencoded({ extended: false }));
app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false
}));
app.use(passport.initialize());
app.use(passport.session());

// Routes
app.post('/register', async (req, res) => {
  try {
    const { username, password } = req.body;
    await User.register(new User({ username }), password);
    res.redirect('/login');
  } catch (err) {
    res.status(500).send(err.message);
  }
});

app.post('/login',
  passport.authenticate('local', {
    successRedirect: '/profile',
    failureRedirect: '/login'
  })
);

app.get('/profile', (req, res) => {
  if (req.isAuthenticated()) {
    res.send(`Chào ${req.user.username}`);
  } else {
    res.redirect('/login');
  }
});

app.listen(3000, () => console.log('Server chạy ở port 3000'));
```

Ví dụ minh họa đầy đủ

Đây là ví dụ ngắn gọn nhất nhưng vẫn đầy đủ chức năng: Đăng ký, Đăng nhập, Kiểm tra trạng thái đăng nhập.

js

```
// models/User.js
```

```

const mongoose = require('mongoose');
const passportLocalMongoose = require('passport-local-mongoose');

const userSchema = new mongoose.Schema({});
userSchema.plugin(passportLocalMongoose);

module.exports = mongoose.model('User', userSchema);

```

js

// config/passport.js

```

const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const User = require('../models/User');

passport.use(new LocalStrategy(User.authenticate()));
passport.serializeUser(User.serializeUser());
passport.deserializeUser(User.deserializeUser());

```

js

// app.js

```

const express = require('express');
const mongoose = require('mongoose');
const session = require('express-session');
const passport = require('passport');
const User = require('./models/User');

const app = express();

mongoose.connect('mongodb://localhost:27017/authDemo', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

app.use(express.urlencoded({ extended: false }));
app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false
}));
app.use(passport.initialize());
app.use(passport.session());

// Route đăng ký
app.post('/register', async (req, res) => {
  const { username, password } = req.body;
  try {
    await User.register(new User({ username }), password);
    res.send('Đăng ký thành công!');
  } catch (err) {
    res.status(500).send(err.message);
  }
});

// Route đăng nhập
app.post('/login',
  passport.authenticate('local', { failureRedirect: '/login' }),
  (req, res) => {
    res.send('Đăng nhập thành công!');
  }
);

// Route cá nhân
app.get('/profile', (req, res) => {
  if (req.isAuthenticated()) {
    res.send(`Xin chào, ${req.user.username}`);
  } else {
    res.send('Bạn chưa đăng nhập');
  }
});

```

```

}
});

app.listen(3000, () => console.log('Server đang chạy ở port 3000'));

```

⚠️ Lỗi thường gặp và cách khắc phục

Không tìm thấy <code>User.register</code>	Chưa import model hoặc không dùng đúng method từ plugin	Dùng <code>User.register()</code> như hướng dẫn
Đăng nhập luôn thất bại	Mật khẩu chưa được hash	Đảm bảo dùng <code>User.register()</code> để tạo tài khoản
Session không hoạt động	Thiếu middleware <code>passport.session()</code>	Thêm <code>app.use(passport.session())</code> sau session middleware

✓ Best Practices

- Luôn sử dụng `bcrypt` hoặc tương tự để hash mật khẩu – `passport-local-mongoose` đã làm điều này sẵn.
- Bảo vệ route bằng middleware kiểm tra `req.isAuthenticated()`.
- Xóa session khi logout để tránh rò rỉ thông tin.
- Sử dụng store bền vững như Redis nếu chuyển sang production.

🔗 Liên kết với các kỹ thuật khác

Passport.js + Local Strategy	<code>passport-local-mongoose</code> là plugin mở rộng của Passport, giúp tích hợp local strategy dễ dàng hơn
Session-based Authentication	Dùng chung với <code>express-session</code> để duy trì trạng thái đăng nhập
JWT / Token-based Auth	Nếu chuyển sang API backend hoặc mobile app, có thể thay thế bằng JWT

🔗 Khi nào nên dùng `passport-local-mongoose`?

- Khi xây dựng ứng dụng web truyền thống có giao diện người dùng (ví dụ: blog, forum).
- Khi muốn nhanh chóng tích hợp xác thực local mà không phải viết nhiều logic hash/match password.
- Khi đang dùng Mongoose làm ODM chính.

📌 Bài học rút ra cho sinh viên:

“Không bao giờ lưu mật khẩu dưới dạng thô. Hãy luôn sử dụng các thư viện chuyên dụng như `passport-local-mongoose` hoặc `bcrypt` để đảm bảo tính bảo mật. Đồng thời, nhớ rằng xác thực chỉ là một phần trong chuỗi bảo mật tổng thể – nó cần đi kèm với phân quyền, kiểm soát phiên và bảo vệ dữ liệu nhạy cảm.”

🔒 Kỹ thuật 29.2: Sử dụng Passport với Session để thiết lập xác thực người dùng

📌 Mô tả

Đây là kỹ thuật phổ biến trong ứng dụng web truyền thống (ví dụ: website có giao diện người dùng). Sau khi người dùng đăng nhập thành công bằng `passport-local`, hệ thống sẽ lưu trạng thái đăng nhập vào **session** và duy trì nó qua nhiều request.

🔗 Khái niệm & Vai trò

- **Session** : Là cơ chế lưu trữ tạm thời thông tin người dùng giữa các request.
- **Passport + session** : Giúp duy trì trạng thái đăng nhập người dùng trên toàn bộ ứng dụng.
- **Vai trò trong bảo mật:**

- Ngăn chặn truy cập trái phép đến các route cần xác thực.
- Đảm bảo rằng chỉ người dùng đã đăng nhập mới được truy cập nội dung nhạy cảm.

Các bước triển khai

Bước 1: Cài đặt thư viện cần thiết

bash

```
npm install express-session passport passport-local mongoose passport-local-mongoose
```

Bước 2: Cấu hình Express + Session

```
js
const express = require('express');
const session = require('express-session');
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const mongoose = require('mongoose');
const User = require('./models/User');

const app = express();

// Kết nối MongoDB
mongoose.connect('mongodb://localhost:27017/authDemo', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

// Middleware
app.use(express.urlencoded({ extended: false }));
app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false
}));
app.use(passport.initialize());
app.use(passport.session());
```

Bước 3: Cấu hình serialize/deserialize user

```
js
passport.serializeUser(function(user, done) {
  done(null, user.id);
});

passport.deserializeUser(function(id, done) {
  User.findById(id, function(err, user) {
    done(err, user);
  });
});
```

Bước 4: Cấu hình Local Strategy

```
js
passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username }, function(err, user) {
      if (err) return done(err);
      if (!user) return done(null, false, { message: 'Không tìm thấy tài khoản' });
      user.comparePassword(password, function(err, isMatch) {
        if (err) return done(err);
        if (isMatch) return done(null, user);
        return done(null, false, { message: 'Mật khẩu không đúng' });
      });
    });
  }
));
```

Bước 5: Tạo route login và protected route

```
js
app.post('/login',
passport.authenticate('local', { failureRedirect: '/login' })),
function(req, res) {
res.redirect('/profile');
}
);

app.get('/profile', ensureAuthenticated, function(req, res) {
res.send(`Xin chào ${req.user.username}`);
});

function ensureAuthenticated(req, res, next) {
if (req.isAuthenticated()) return next();
res.redirect('/login');
}
```

Ví dụ đầy đủ đơn giản (User Model)

```
js
// models/User.js
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const passportLocalMongoose = require('passport-local-mongoose');

const UserSchema = new Schema({
username: String,
password: String
});

UserSchema.plugin(passportLocalMongoose);

module.exports = mongoose.model('User', UserSchema);
```

Lỗi thường gặp

Không đăng nhập được sau khi reload	Thiếu <code>serializeUser / deserializeUser</code>	Thêm middleware xử lý session
Đăng nhập xong vẫn bị redirect về login	Middleware chưa nhận ra phiên đăng nhập	Kiểm tra thứ tự <code>passport.session()</code>
Session không lưu vào DB	Dùng <code>MemoryStore</code> mặc định	Sử dụng <code>connect-mongo</code> hoặc <code>Redis</code>

Best Practices

- Luôn sử dụng store bền vững như **MongoDB** hoặc **Redis** cho production.
- Đặt thời gian sống cho session (`maxAge`) hợp lý.
- Xóa session khi logout để tránh rò rỉ thông tin.
- Bảo vệ cookie bằng thuộc tính `secure`, `httpOnly`.

Liên kết với các kỹ thuật khác

- **Dùng chung với `passport-local` và `passport-local-mongoose`** : xác thực local + hash mật khẩu + session.
- **Kết hợp với JWT** nếu chuyển sang API backend – nhưng nên chọn một mô hình phù hợp với từng dự án.

Kỹ thuật 30: Token-based Authentication (JWT)

Mô tả

JWT (JSON Web Token) là phương pháp xác thực không dùng session, rất phù hợp với ứng dụng RESTful API, mobile app và SPA (Single Page Application).

Khái niệm & Vai trò

- **JWT** : Chuỗi mã hóa chứa payload (thông tin người dùng), chữ ký số để đảm bảo tính toàn vẹn.
- **Stateless authentication** : Server không lưu trữ trạng thái người dùng → dễ mở rộng.
- **Vai trò trong bảo mật**:
 - Chống CSRF, phù hợp với ứng dụng đa nền tảng.
 - Dễ tích hợp với microservices và API gateway.
 - Có thể kiểm soát thời gian sống của token.

Các bước triển khai

Bước 1: Cài đặt thư viện cần thiết

bash

1

```
npm install jsonwebtoken bcryptjs express-jwt
```

Bước 2: Tạo token sau khi đăng nhập

js

```
const jwt = require('jsonwebtoken');

app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });

  if (!user || !user.comparePassword(password)) {
    return res.status(401).json({ message: 'Tài khoản hoặc mật khẩu sai' });
  }

  const token = jwt.sign({ id: user._id, username: user.username }, 'secret-key', {
    expiresIn: '1h'
  });

  res.json({ token });
});
```

Bước 3: Bảo vệ route bằng middleware verify token

js

```
const expressJwt = require('express-jwt');

app.get('/profile', expressJwt({ secret: 'secret-key', algorithms: ['HS256'] }), (req, res) => {
  res.json(req.auth); // Thông tin từ token
});
```

Ví dụ đầy đủ (tích hợp với User model)

js

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema({
  username: String,
  password: String
});

UserSchema.methods.comparePassword = function(candidatePassword, callback) {
  bcrypt.compare(candidatePassword, this.password, function(err, isMatch) {
```

```

callback(err, isMatch);
});
};

module.exports = mongoose.model('User', UserSchema);

```

⚠ Lỗi thường gặp

Token hết hạn nhưng vẫn dùng được	Không xử lý lỗi expired token	Dùng try/catch khi verify
Token bị lộ ngoài client	Gửi qua HTTP không bảo mật	Dùng HTTPS bắt buộc
Không kiểm tra hết hạn token	Dùng sign mà không expiresIn	Thêm option thời gian sống

✓ Best Practices

- Luôn dùng HTTPS để truyền token.
- Chỉ cấp token có thời gian sống ngắn, dùng refresh token để gia hạn.
- Lưu trữ token ở phía client bằng `HttpOnly Cookie` hoặc `localStorage` tùy ngữ cảnh.
- Sử dụng `refresh token blacklist` để ngăn token cũ sau khi logout.

🔗 Liên kết với các kỹ thuật khác

- **Kết hợp với Passport (JWT Strategy)** : thay vì `passport-local`.
- **Kết hợp với OAuth2 hoặc OpenID Connect** trong các ứng dụng quy mô lớn.
- **Refresh token mechanism** để tăng tính bảo mật và trải nghiệm người dùng.

🔗 Tổng kết liên kết giữa các kỹ thuật

Passport-Local	Xác thực local bằng email/password	Web app truyền thống	Passport-Local-Mongoose, Session
Passport-Local-Mongoose	Mã hóa mật khẩu tự động	Web app có xác thực local	Passport, Session
Passport + Session	Duy trì trạng thái đăng nhập	Web app có giao diện người dùng	Passport-Local, Local-Mongoose
JWT	Xác thực stateless cho API	Mobile App, REST API, SPA	Refresh Token, Bcrypt

KINH NGHIỆM THỰC TIỄN

Kỹ thuật 28: User Authentication With Passport + Passport-Local

✓ Kinh nghiệm thực tế khi sử dụng trong dự án Node.js

Điều gì phát sinh ngoài lý thuyết?

- Trong thực tế, không phải tất cả người dùng đều có username – một số hệ thống dùng email thay thế.
- Người dùng quên mật khẩu → cần thêm chức năng reset password (không thuộc scope passport-local).
- Nhiều ứng dụng kết hợp local login với OAuth (Google/Facebook), đòi hỏi quản lý nhiều loại tài khoản cùng lúc.

Tình huống cụ thể:

Một ứng dụng web cho phép đăng nhập bằng email/password hoặc Google. Nếu không xử lý đồng bộ giữa hai cách xác thực, người dùng có thể bị tạo ra nhiều bản ghi khác nhau trong DB dù là cùng một người.

Lỗi thường gặp khi lập trình hoặc tích hợp

req.user luôn là undefined	Thiếu middleware <code>passport.session()</code>	Dù đăng nhập thành công nhưng vẫn không thấy <code>req.user</code>	Thêm <code>app.use(passport.session())</code> sau session middleware
Không redirect sau khi đăng nhập	Chưa trả response đúng cách trong route handler	Đăng nhập xong mà không redirect hay phản hồi gì	Dùng <code>res.redirect()</code> hoặc <code>res.json()</code> tùy frontend
Sai logic verify function	So sánh mật khẩu sai hoặc không gọi <code>done()</code>	Người dùng đúng nhưng không đăng nhập được	Kiểm tra hàm <code>comparePassword</code> , đảm bảo gọi <code>done()</code>

Mẹo lập trình giúp sinh viên tiết kiệm thời gian

- Tạo file cấu hình riêng cho Passport (`config/passport.js`) để dễ bảo trì.
- Viết hàm middleware kiểm tra đăng nhập chung:

js

```
function ensureAuthenticated(req, res, next) {  
  if (req.isAuthenticated()) return next();  
  res.status(401).send('Bạn chưa đăng nhập');  
}
```

- Gắn middleware này vào các route cần bảo vệ.

Best Practices

Bảo mật mật khẩu	Luôn hash mật khẩu trước khi lưu – nên dùng <code>passport-local-mongoose</code> hoặc <code>bcrypt</code>
Cấu trúc thư mục	Phân tách model, config, route, controller để dễ mở rộng
Middleware	Đặt thứ tự đúng: <code>session</code> → <code>passport.initialize()</code> → <code>passport.session()</code>
Xử lý lỗi	Bắt lỗi trong <code>async/await</code> hoặc dùng <code>try/catch</code> wrapper
HTTPS	Luôn dùng HTTPS trong môi trường production để truyền thông tin đăng nhập an toàn

Liên hệ với trải nghiệm thực tế

Trong một dự án website bán hàng tại Việt Nam, nhóm phát triển gặp vấn đề vì quên thêm `passport.session()` → người dùng đăng nhập xong nhưng không duy trì trạng thái. Sau khi debug, nhóm nhận ra rằng Passport chỉ xác thực ở mỗi request nếu không có session middleware.

Kỹ thuật 29.1: Sử dụng passport-local-mongoose để thiết lập xác thực local

Kinh nghiệm thực tế khi sử dụng trong dự án Node.js

Điều gì phát sinh ngoài lý thuyết?

- `passport-local-mongoose` mặc định dùng field `username`. Nếu muốn dùng `email` thay thế, cần cấu hình lại:

js

```
userSchema.plugin(passportLocalMongoose, { usernameField: 'email' });
```

- Một số ứng dụng yêu cầu xác thực email trước khi kích hoạt tài khoản – điều này không nằm trong scope của plugin.

Tình huống cụ thể:

Trong một dự án forum, sinh viên quên cấu hình lại `usernameField` sang `email`, khiến ứng dụng không thể đăng nhập bằng email như mong muốn.

Lỗi thường gặp khi lập trình hoặc tích hợp

User.register is not a function	Không import đúng model hoặc không apply plugin	Gọi <code>User.register(...)</code> nhưng báo lỗi	Kiểm tra lại schema đã cài plugin chưa
Mật khẩu không được hash	Dùng <code>new User({...})</code> rồi <code>.save()</code> thay vì <code>User.register()</code>	Mật khẩu lưu dưới dạng plain text	Dùng <code>User.register(new User({ username }), password)</code>
Đăng ký thành công nhưng không đăng nhập được	Mật khẩu bị sai do typo hoặc không match	Người dùng chắc chắn đúng nhưng vẫn không vào được	Log mật khẩu từ DB ra, so sánh với input

Mẹo lập trình giúp sinh viên tiết kiệm thời gian

- Tự động validate dữ liệu đầu vào trước khi gọi `User.register()`:

js

```
if (!username || !password) {  
  return res.status(400).send('Thiếu tên đăng nhập hoặc mật khẩu');  
}
```

- Dùng `console.log(req.body)` hoặc Postman để test API trước khi frontend gửi request.
- Tạo script seed database với vài user mẫu để test nhanh.

Best Practices

Hash mật khẩu	Dùng <code>passport-local-mongoose</code> hoặc <code>bcrypt</code> – không bao giờ lưu mật khẩu raw
Validate đầu vào	Kiểm tra dữ liệu trước khi đưa vào DB
Schema linh hoạt	Có thể mở rộng schema để chứa thêm <code>email</code> , <code>fullname</code> , <code>avatarUrl</code> , v.v.
Test API độc lập	Dùng Postman hoặc Thunder Client để kiểm thử endpoint trước khi frontend tích hợp
Bảo vệ route	Luôn check <code>req.isAuthenticated()</code> trước khi trả data cá nhân

Liên hệ với trải nghiệm thực tế

Một startup xây dựng app blog cá nhân đã dùng `passport-local-mongoose` để nhanh chóng triển khai xác thực local, sau đó nâng cấp lên OAuth2 để hỗ trợ đăng nhập bằng Google. Nhờ schema đơn giản và logic rõ ràng, quá trình nâng cấp diễn ra thuận lợi.

Kỹ thuật 30: Token-based Authentication (JWT)

Kinh nghiệm thực tế khi sử dụng trong dự án Node.js

Điều gì phát sinh ngoài lý thuyết?

- Token hết hạn nhưng client vẫn dùng → cần xử lý error code 401 và refresh token.
- JWT không phù hợp với ứng dụng cần logout tức thì – cần blacklist token hoặc dùng Redis.

Tình huống cụ thể:

Trong một ứng dụng mobile, người dùng đăng xuất nhưng token vẫn còn hiệu lực đến hết giờ. Hacker có thể dùng token cũ để truy cập tài khoản nếu không có cơ chế blacklisting.

Lỗi thường gặp khi lập trình hoặc tích hợp

Token không có chữ ký	Quên secret key khi sign hoặc verify	<code>jwt.sign(...)</code> thiếu secret → token không hợp lệ	Dùng biến môi trường <code>process.env.JWT_SECRET</code>
Token bị lộ	Gửi qua HTTP thay vì HTTPS	Hacker đọc được token trong network log	Dùng HTTPS bắt buộc
Không bắt lỗi verify	Token hết hạn hoặc bị sửa đổi → server crash	<code>jwt.verify(token, secret)</code> không try/catch	Dùng try/catch hoặc middleware custom

Mẹo lập trình giúp sinh viên tiết kiệm thời gian

- Tạo file `utils/auth.js` chứa các hàm tiện ích như:

```
js
function generateToken(user) {
  return jwt.sign({ id: user._id, username: user.username }, process.env.JWT_SECRET, {
    expiresIn: '1h' });
}
```

- Dùng middleware để kiểm tra token:

```
js
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
}
```

Best Practices

Thời gian sống token	Dùng <code>expiresIn</code> ngắn (ví dụ: 15 phút - 1 giờ)
Lưu trữ token	Nên dùng HttpOnly Cookie hoặc localStorage tùy ngữ cảnh
Secret Key	Lưu trong biến môi trường, không hardcode trong code
Blacklist token	Dùng Redis để lưu token đã hủy khi logout
HTTPS	Luôn bật HTTPS khi truyền token qua mạng

Liên hệ với trải nghiệm thực tế

Một startup fintech dùng JWT để xác thực API backend cho ứng dụng di động. Họ gặp vấn đề về logout không tức thì, nên đã tích hợp Redis để lưu trữ danh sách token bị hủy. Giải pháp này giúp tăng tính bảo mật và trải nghiệm người dùng.

Tổng kết Best Practices chung cho cả 3 kỹ thuật

Mã hóa mật khẩu	Dùng <code>bcrypt</code> , <code>argon2</code> , hoặc <code>passport-local-mongoose</code>
Cấu hình session	Sử dụng store bền vững như Redis hoặc MongoDB
Token	Dùng JWT với thời gian sống hợp lý, kèm cơ chế refresh token
Bảo mật tổng thể	Luôn dùng HTTPS, validate input, log lỗi đầy đủ
Tổ chức code	Tách module: models, routes, controllers, config, utils

EXERCISE

Tên Dự Án:

Auth System – Hệ Thống Xác Thực Đa Nền Tảng

Một hệ thống backend RESTful API cho phép người dùng đăng ký, đăng nhập bằng tài khoản local (email/password) hoặc token (JWT), hỗ trợ phân quyền giữa user và admin.

Mục tiêu bài học

- Hiểu và áp dụng thành thạo các kỹ thuật xác thực trong Node.js.
- Biết cách tổ chức một ứng dụng có tính bảo mật cao với Passport, JWT, Session.
- Có khả năng mở rộng sang OAuth, multi-factor authentication trong tương lai.

Mục tiêu dự án

- Đăng ký, đăng nhập bằng email/password.
- Hỗ trợ xác thực bằng session và JWT.
- Phân quyền giữa user và admin.
- Bảo vệ route dựa trên trạng thái đăng nhập và vai trò người dùng.
- Tổ chức code theo module để dễ bảo trì và mở rộng.

Công nghệ sử dụng

Backend	Node.js + Express
Cơ sở dữ liệu	MongoDB (Mongoose)
Xác thực	Passport.js, passport-local, passport-local-mongoose
Token	JSON Web Token (jsonwebtoken)
Session	express-session
Middleware	dotenv, morgan, helmet
Kiểm thử	Postman / Thunder Client
Môi trường	VSCode, Git, npm/yarn

Cấu trúc thư mục gợi ý

```
auth-system/
|
├── config/ # Cấu hình database, passport
|   ├── db.js
|   └── passport.js
|
├── models/ # Schema User
|   └── User.js
|
├── routes/ # Các route API và web
|   ├── authRoutes.js
|   └── protectedRoutes.js
|
├── controllers/ # Controller xử lý logic
|   ├── authController.js
|   └── userController.js
|
├── middleware/ # Middleware custom
|   ├── authMiddleware.js
|   └── jwtMiddleware.js
|
```

```
├── utils/ # Helper functions
|   └── jwtUtils.js
|
├── .env # Biến môi trường
├── app.js # File chính
└── package.json
```

✓ Tóm tắt danh sách các bước thực hiện

1. Khởi tạo project và cài đặt dependencies.
2. Thiết lập kết nối MongoDB.
3. Tạo schema User với passport-local-mongoose.
4. Cấu hình Passport và session.
5. Viết controller và route cho register/login/logout.
6. Bảo vệ route bằng middleware kiểm tra đăng nhập.
7. Thêm JWT để hỗ trợ API backend.
8. Viết middleware kiểm tra role.
9. Test API bằng Postman.
10. Tạo database mẫu trong MongoDB Compass.
11. Deploy (tùy chọn).

Chi tiết từng bước thực hiện

Bước 1: Khởi tạo project và cài đặt dependencies

bash

```
mkdir auth-system
cd auth-system
npm init -y
```

```
npm install express mongoose dotenv morgan helmet cors
npm install passport passport-local passport-local-mongoose express-session
npm install jsonwebtoken bcryptjs
```

Bước 2: Thiết lập kết nối MongoDB

js

// config/db.js

```
// config/db.js
const mongoose = require('mongoose');

module.exports = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true
      // Removed useCreateIndex and useFindAndModify as they are
      deprecated in Mongoose 6+
    });
    console.log('Kết nối DB thành công');
  } catch (err) {
    console.error('Lỗi kết nối DB:', err.message);
    process.exit(1); // Exit process with failure
  }
};
```

Bước 3: Tạo schema User với passport-local-mongoose

js

// models/User.js

```
// models/User.js
const mongoose = require('mongoose');
const passportLocalMongoose = require('passport-local-mongoose');

// Define the User schema
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true, // Ensure email is unique
    lowercase: true, // Store emails in lowercase
    trim: true // Remove whitespace from both ends
  },
  role: {
    type: String,
    enum: ['user', 'admin'], // Role can only be 'user' or 'admin'
    default: 'user' // Default role is 'user'
  },
  createdAt: {
    type: Date,
    default: Date.now // Automatically set creation date
  }
});

// Apply the passport-local-mongoose plugin to the user schema
// This plugin handles password hashing and adds methods for authentication
userSchema.plugin(passportLocalMongoose, {
  usernameField: 'email' // Tell passport-local-mongoose to use 'email' as the
  username field
});

// Export the User model
module.exports = mongoose.model('User', userSchema);
```

Bước 4: Cấu hình Passport và session

js

// config/passport.js

```
// config/passport.js
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const User = require('../models/User'); // Import the User model

// Use the local strategy provided by passport-local-mongoose
// passport-local-mongoose adds `authenticate` method to User model
passport.use(new LocalStrategy({
  usernameField: 'email', // Specify 'email' as the field for username
  passwordField: 'password' // Specify 'password' as the field for password
```

```

}, User.authenticate()));

// Serialize user into the session
// passport-local-mongoose adds `serializeUser` method to User model
passport.serializeUser(User.serializeUser());

// Deserialize user from the session
// passport-local-mongoose adds `deserializeUser` method to User model
passport.deserializeUser(User.deserializeUser());

console.log('✅ Passport configured successfully');

```

js

// app.js

```

require('dotenv').config(); // Load environment variables from .env file
const express = require('express');
const mongoose = require('mongoose');
const session = require('express-session');
const passport = require('passport');
const morgan = require('morgan'); // HTTP request logger middleware
const helmet = require('helmet'); // Security middleware
const cors = require('cors'); // Enable CORS

// Import configuration and routes
const connectDB = require('./config/db');
require('./config/passport'); // Configure Passport strategies
const authRoutes = require('./routes/authRoutes');
const protectedRoutes = require('./routes/protectedRoutes');

const app = express();

// Connect to MongoDB
connectDB();

// Middleware
app.use(morgan('dev')); // Log HTTP requests to the console
app.use(helmet()); // Set various HTTP headers for security
app.use(cors({
  origin: 'http://localhost:3000', // Allow requests from this origin (adjust
  // as needed for your frontend)
  credentials: true // Allow cookies to be sent
}));
app.use(express.json()); // Parse JSON request bodies
app.use(express.urlencoded({ extended: true })); // Parse URL-encoded request
// bodies

// Session middleware configuration
app.use(session({
  secret: process.env.SESSION_SECRET, // Secret key for signing the session ID
  cookie

```

```

    resave: false, // Don't save session if unmodified
    saveUninitialized: false, // Don't create session until something is stored
    cookie: {
      httpOnly: true, // Prevents client-side JS from accessing the cookie
      secure: process.env.NODE_ENV === 'production', // Only send cookie over
HTTPS in production
      maxAge: 24 * 60 * 60 * 1000 // 1 day
    }
  }
}));

// Passport initialization
app.use(passport.initialize());
app.use(passport.session()); // Enable session support for Passport

// Basic route for testing
app.get('/', (req, res) => {
  res.send('Welcome to the Auth System API!');
});

// Route handlers
app.use('/api/auth', authRoutes); // Authentication routes (register, login,
logout, login-jwt)
app.use('/api', protectedRoutes); // Protected routes

// Global error handling middleware (should be the last middleware)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!', details: err.message
});
});

// Start the server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
  console.log(`Access the API at http://localhost:${PORT}`);
});

```

Bước 5: Viết controller và route

js

// controllers/authController.js

```

// controllers/authController.js
const User = require('../models/User'); // Import the User model
const jwtUtils = require('../utils/jwtUtils'); // Import JWT utility functions

/**
 * Handles user registration.
 * Registers a new user with email and password.
 */

```

```

exports.register = async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ error: 'Email và mật khẩu là bắt buộc.' });
  }
  try {
    // User.register is a static method provided by passport-local-mongoose
    // It hashes the password and saves the user to the database
    await User.register(new User({ email }), password);
    res.status(201).json({ message: 'Đăng ký tài khoản thành công.' });
  } catch (err) {
    // Handle duplicate email error
    if (err.name === 'UserExistsError') {
      return res.status(409).json({ error: 'Email đã tồn tại. Vui lòng sử dụng email khác.' });
    }
    console.error('Lỗi đăng ký:', err);
    res.status(500).json({ error: 'Đăng ký thất bại. Vui lòng thử lại.',
      details: err.message });
  }
};

/**
 * Handles user login for session-based authentication.
 * This function is called after passport.authenticate('local') has successfully
 * authenticated the user.
 * req.user will contain the authenticated user object.
 */
exports.login = (req, res) => {
  // If we reach here, authentication was successful via session
  res.json({
    message: 'Đăng nhập thành công (Session).',
    user: {
      id: req.user._id,
      email: req.user.email,
      role: req.user.role
    }
  });
};

/**
 * Handles user login for JWT-based authentication.
 * This function is called after passport.authenticate('local', { session: false
 * })
 * has successfully authenticated the user without creating a session.
 * req.user will contain the authenticated user object.
 */
exports.loginJwt = (req, res) => {
  // If we reach here, authentication was successful (without session)
  // Generate a JWT token for the authenticated user

```

```

const token = jwtUtils.generateToken(req.user);
res.json({
  message: 'Đăng nhập thành công (JWT).',
  token, // Send the JWT token to the client
  user: {
    id: req.user._id,
    email: req.user.email,
    role: req.user.role
  }
});
});

/**
 * Handles user logout for session-based authentication.
 * Removes the user from the session.
 */
exports.logout = (req, res, next) => {
  // req.logout is provided by Passport to terminate a login session
  req.logout((err) => {
    if (err) {
      console.error('Lỗi đăng xuất:', err);
      return next(err); // Pass error to the error handling middleware
    }
    res.json({ message: 'Đăng xuất thành công.' });
  });
};

```

//routes/authRoutes.js

```

// routes/authRoutes.js
const express = require('express');
const router = express.Router();
const passport = require('passport'); // Import Passport for authentication
const authController = require('../controllers/authController'); // Import auth controller

// POST /api/auth/register - Register a new user
router.post('/register', authController.register);

// POST /api/auth/login - Login using session-based authentication
// passport.authenticate('local') middleware handles authentication
router.post('/login',
  passport.authenticate('local', {
    failureFlash: false // Set to true if you use connect-flash for flash messages
  }),
  authController.login // If authentication succeeds, this controller handles the response
);

```

```
// POST /api/auth/login-jwt - Login using JWT-based authentication (no session)
// passport.authenticate('local', { session: false }) ensures no session is
// created
router.post('/login-jwt',
  passport.authenticate('local', { session: false }),
  authController.loginJwt // If authentication succeeds, this controller
// generates and sends JWT
);

// GET /api/auth/logout - Logout the user (for session-based logout)
router.get('/logout', authController.logout);

module.exports = router;
```

// routes/protectedRoutes.js

```
// routes/protectedRoutes.js
const express = require('express');
const router = express.Router();
const { ensureAuthenticated, ensureAdmin } =
require('../middleware/authMiddleware'); // For session-based protection
const { authenticateJwt } = require('../middleware/jwtMiddleware'); // For JWT-
// based protection
const userController = require('../controllers/userController'); // Import user
// controller

// Example route protected by session authentication
// GET /api/profile - Get user profile (requires session login)
router.get('/profile', ensureAuthenticated, (req, res) => {
  // req.user is available here due to passport.session()
  res.json({
    message: 'Bạn đã đăng nhập bằng session.',
    user: {
      id: req.user._id,
      email: req.user.email,
      role: req.user.role
    }
  });
});

// Example route protected by session and role check
// GET /api/admin - Admin-only route (requires session login and admin role)
router.get('/admin', ensureAdmin, (req, res) => {
  res.json({ message: 'Chào mừng Admin! Đây là thông tin chỉ dành cho Admin.'
});
});

// Example route protected by JWT authentication
// GET /api/jwt-profile - Get user profile (requires valid JWT token)
router.get('/jwt-profile', authenticateJwt, (req, res) => {
```

```

    // req.user is available here due to jwtMiddleware
    res.json({
      message: 'Bạn đã xác thực bằng JWT.',
      user: {
        id: req.user.id, // ID from JWT payload
        email: req.user.email, // Email from JWT payload
        role: req.user.role // Role from JWT payload
      }
    });
  });

// Example route protected by JWT and role check
// GET /api/jwt-admin - Admin-only route (requires valid JWT token and admin role)
router.get('/jwt-admin', authenticateJwt, (req, res) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Truy cập bị từ chối: Không có quyền Admin.' });
  }
  res.json({ message: 'Chào mừng Admin! Đây là thông tin chỉ dành cho Admin (qua JWT).' });
});

// Get all users (for admin - demonstrating userController usage)
router.get('/users', authenticateJwt, ensureAdmin, userController.getAllUsers);

module.exports = router;

```

Bước 6: Bảo vệ route bằng middleware kiểm tra đăng nhập

js

// middleware/authMiddleware.js

```

// middleware/authMiddleware.js

/**
 * Middleware to ensure the user is authenticated (logged in via session).
 * If authenticated, proceeds to the next middleware/route handler.
 * Otherwise, sends a 401 Unauthorized response.
 */
exports.ensureAuthenticated = (req, res, next) => {
  if (req.isAuthenticated()) {
    // req.isAuthenticated() is a Passport method that returns true if the
    user is authenticated
    return next();
  }
  res.status(401).json({ error: 'Bạn chưa đăng nhập. Vui lòng đăng nhập để truy cập.' });
};

/**

```

```

    * Middleware to ensure the user has 'admin' role (for session-based authentication).
    * Requires the user to be authenticated first.
    * If the user is an admin, proceeds. Otherwise, sends a 403 Forbidden response.
    */
exports.ensureAdmin = (req, res, next) => {
    // First, ensure the user is authenticated
    if (!req.isAuthenticated()) {
        return res.status(401).json({ error: 'Bạn chưa đăng nhập.' });
    }
    // Then, check if the user has the 'admin' role
    if (req.user && req.user.role === 'admin') {
        return next();
    }
    res.status(403).json({ error: 'Truy cập bị từ chối: Bạn không có quyền quản trị.' });
};

```

js

// routes/protectedRoutes.js

```

// middleware/jwtMiddleware.js
const jwt = require('jsonwebtoken'); // Import jsonwebtoken library

/**
 * Middleware to authenticate requests using JWT.
 * Expects a JWT in the 'Authorization' header in the format 'Bearer <token>'.
 * If the token is valid, it decodes the user information and attaches it to `req.user`.
 * Otherwise, sends a 401 Unauthorized or 403 Forbidden response.
 */
exports.authenticateJwt = (req, res, next) => {
    // Get the Authorization header
    const authHeader = req.headers['authorization'];

    // Check if the header exists and starts with 'Bearer '
    // Example: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
    const token = authHeader && authHeader.split(' ')[1];

    // If no token is provided, return 401 Unauthorized
    if (!token) {
        return res.status(401).json({ error: 'Không có token xác thực được cung cấp.' });
    }

    // Verify the token using the JWT_SECRET
    jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
        if (err) {
            // If verification fails (e.g., token expired, invalid signature),
            return 403 Forbidden
            console.error('Lỗi xác thực JWT:', err.message);
        }
    });
};

```

```

        return res.status(403).json({ error: 'Token không hợp lệ hoặc đã hết hạn.' });
    }
    // If the token is valid, attach the decoded user payload to req.user
    // The 'user' here is the payload we signed into the token
    req.user = user;
    next(); // Proceed to the next middleware or route handler
  });
};

```

Bước 7: Thêm JWT để hỗ trợ API backend

js

// utils/jwtUtils.js

```

// utils/jwtUtils.js
const jwt = require('jsonwebtoken'); // Import jsonwebtoken library

/**
 * Generates a JSON Web Token (JWT) for the given user.
 * The token contains user's ID, email, and role, and expires in 1 hour.
 * @param {Object} user - The user object from which to create the token payload.
 * @returns {string} The generated JWT.
 */
exports.generateToken = (user) => {
  // Payload for the JWT. Includes sensitive user data that should be minimal.
  // Ensure you only include necessary identifying information.
  const payload = {
    id: user._id, // User ID from MongoDB
    email: user.email,
    role: user.role
  };

  // Sign the token with the secret key from environment variables
  // and set an expiration time.
  return jwt.sign(
    payload,
    process.env.JWT_SECRET, // Secret key from .env file
    { expiresIn: '1h' } // Token expires in 1 hour
  );
};

```

js

// controllers/userController.js

```

// // controllers/userController.js
const User = require('../models/User'); // Import the User model

/**
 * Get all users (Admin only)

```

```

*/
exports.getAllUsers = async (req, res) => {
  try {
    const users = await User.find({}, '-hash -salt'); // Exclude password
    hash and salt
    res.status(200).json({
      message: 'Danh sách tất cả người dùng.',
      users: users.map(user => ({
        id: user._id,
        email: user.email,
        role: user.role,
        createdAt: user.createdAt
      })))
    });
  } catch (err) {
    console.error('Lỗi khi lấy danh sách người dùng:', err);
    res.status(500).json({ error: 'Không thể lấy danh sách người dùng.',
    details: err.message });
  }
};

```

js

// middleware/jwtMiddleware.js

```

// middleware/jwtMiddleware.js
const jwt = require('jsonwebtoken'); // Import jsonwebtoken library

/**
 * Middleware to authenticate requests using JWT.
 * Expects a JWT in the 'Authorization' header in the format 'Bearer <token>'.
 * If the token is valid, it decodes the user information and attaches it to
 * `req.user`.
 * Otherwise, sends a 401 Unauthorized or 403 Forbidden response.
 */
exports.authenticateJwt = (req, res, next) => {
  // Get the Authorization header
  const authHeader = req.headers['authorization'];

  // Check if the header exists and starts with 'Bearer '
  // Example: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
  const token = authHeader && authHeader.split(' ')[1];

  // If no token is provided, return 401 Unauthorized
  if (!token) {
    return res.status(401).json({ error: 'Không có token xác thực được cung
    cấp.' });
  }

  // Verify the token using the JWT_SECRET
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {

```

```

    if (err) {
        // If verification fails (e.g., token expired, invalid signature),
        return 403 Forbidden
        console.error('Lỗi xác thực JWT:', err.message);
        return res.status(403).json({ error: 'Token không hợp lệ hoặc đã hết hạn.' });
    }
    // If the token is valid, attach the decoded user payload to req.user
    // The 'user' here is the payload we signed into the token
    req.user = user;
    next(); // Proceed to the next middleware or route handler
});
};

```

Testcase (05 testcase)

1	Đăng ký tài khoản mới	{ "email": " user@example.com ", "password": "123456" }	Trả về success
2	Đăng nhập thành công	{ "email": " user@example.com ", "password": "123456" }	Trả về user info
3	Truy cập route /profile khi chưa đăng nhập	GET /profile	401 Unauthorized
4	Truy cập route /admin với user thường	GET /admin	403 Forbidden
5	Gửi token hết hạn lên /profile	Bearer <token cũ>	403 Forbidden

Hướng dẫn cấu hình

1. Tạo file .env:

```

env
PORT=3000
SESSION_SECRET=my-secret-key-for-session
JWT_SECRET=my-secret-key-for-jwt

```

Hướng dẫn tạo dữ liệu mẫu trong MongoDB Compass

Bước 1: Mở MongoDB Compass

- Kết nối tới mongodb://localhost:27017

Bước 2: Tạo database authDemo và collection users

Bước 3: Thêm dữ liệu mẫu

Tài khoản người dùng thường

```

json
{
  "email": "user@example.com",
  "password": "$2a$10$QFw7qkPZiWZzjXoBZlYgCeRvGJxN4E9TbIuUOYsPpLZsHJmZy",
  "role": "user",
  "username": "user@example.com"
}

```

Tài khoản admin

```

json
{
  "email": "admin@example.com",
  "password": "$2a$10$QFw7qkPZiWZzjXoBZlYgCeRvGJxN4E9TbIuUOYsPpLZsHJmZy",
  "role": "admin",
  "username": "admin@example.com"
}

```

}

Lưu ý lỗi và tips

User.register is not a function	Chưa apply plugin passport-local-mongoose	Kiểm tra lại model
req.user luôn là undefined	Thiếu <code>passport.session()</code>	Thêm middleware đúng thứ tự
Token không hợp lệ	Quên secret key khi sign hoặc verify	Dùng biến môi trường
Không logout được	Dùng <code>req.logout()</code> mà không callback	Dùng callback như ví dụ
Route không phân quyền	Thiếu middleware kiểm tra role	Thêm middleware đảm bảo

Hướng dẫn tạo secret key cho Session và JWT

Bước 5: Tạo SESSION_SECRET và JWT_SECRET

Bạn có thể tạo thủ công hoặc dùng công cụ sinh key an toàn.

Cách 1: Tự viết một chuỗi bí mật

Ví dụ:

```
env
SESSION_SECRET=my-secret-key-for-session
JWT_SECRET=my-secret-key-for-jwt
```

Cách 2: Dùng Node.js để sinh random string an toàn (tùy chọn)

bash

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

Kết quả:

d8c9a12f3e... (một chuỗi hex dài 128 ký tự)

Sau đó thêm vào file `.env`:

```
env
SESSION_SECRET=d8c9a12f3e...
JWT_SECRET=abc123xyz789...
```

Tips & mẹo xử lý

- Dùng Postman để test endpoint trước khi tích hợp frontend.
- Dùng `dotenv` để quản lý biến môi trường.
- Luôn validate input trước khi xử lý.
- Tách middleware để tái sử dụng.
- Log lỗi đầy đủ để debug nhanh hơn.

Checklist hoàn tất dự án

- Đăng ký thành công
- Đăng nhập bằng Passport + Session
- Đăng nhập bằng JWT
- Logout thành công
- Phân quyền giữa user và admin
- Dùng middleware kiểm tra đăng nhập và vai trò
- Có cấu trúc thư mục rõ ràng
- Đã test qua Postman

Hướng dẫn chạy và test dự án

Hướng dẫn sinh viên hoặc lập trình viên mới bắt đầu:

- Khởi động server thành công.
- Test được các chức năng chính: đăng ký, đăng nhập (session & JWT), phân quyền, đăng xuất.
- Hiểu cách hoạt động của hệ thống xác thực trong thực tế.

Yêu cầu trước khi bắt đầu

1. Đảm bảo bạn đã cài đặt:

- [Node.js](#) (version $\geq 14.x$)
- [MongoDB](#) (local hoặc Atlas)
- [MongoDB Compass](#) (tùy chọn)
- [Postman](#) hoặc Thunder Client (VSCode)

2. Cấu trúc thư mục dự án mẫu:

```
auth-system/  
├── config/  
├── models/  
├── routes/  
├── controllers/  
├── middleware/  
├── utils/  
├── .env  
├── app.js  
└── package.json
```

Bước 1: Cài đặt dependencies

```
bash
```

```
npm install
```

Nếu chưa có file `package.json`, hãy chạy:

```
bash
```

```
npm init -y
```

Sau đó cài các thư viện cần thiết:

```
bash
```

```
npm install express mongoose dotenv morgan helmet cors
```

```
npm install passport passport-local passport-local-mongoose express-session
```

```
npm install jsonwebtoken bcryptjs
```

Bước 2: Tạo file `.env` và cấu hình biến môi trường

Tạo file `.env` ở thư mục gốc với nội dung:

```
env
```

```
PORT=3000
```

```
SESSION_SECRET=894c8a90fc682bb3744b0a98153a496b560b34fe0a95e34e78f4f7e17c1d03d2d477413  
67c6fa97346fd658591676ac85f49cceb9ae050361bed392db2aa3694
```

```
JWT_SECRET=your-jwt-secret-here
```

Bước 3: Chạy MongoDB

Nếu dùng local:

bash

brew services start mongodb-community@6.0 # macOS

sudo service mongodb start # Ubuntu

Kiểm tra kết nối bằng MongoDB Compass nếu muốn.

Bước 4: Khởi động ứng dụng

Chạy lệnh sau trong terminal:

bash

node app.js

Nếu không có lỗi, bạn sẽ thấy thông báo:

Server đang chạy ở port 3000

Kết nối DB thành công

Bước 5: Test API bằng Postman / Thunder Client

Sử dụng Postman hoặc Thunder Client để test các endpoint dưới đây.

1. Đăng ký tài khoản mới

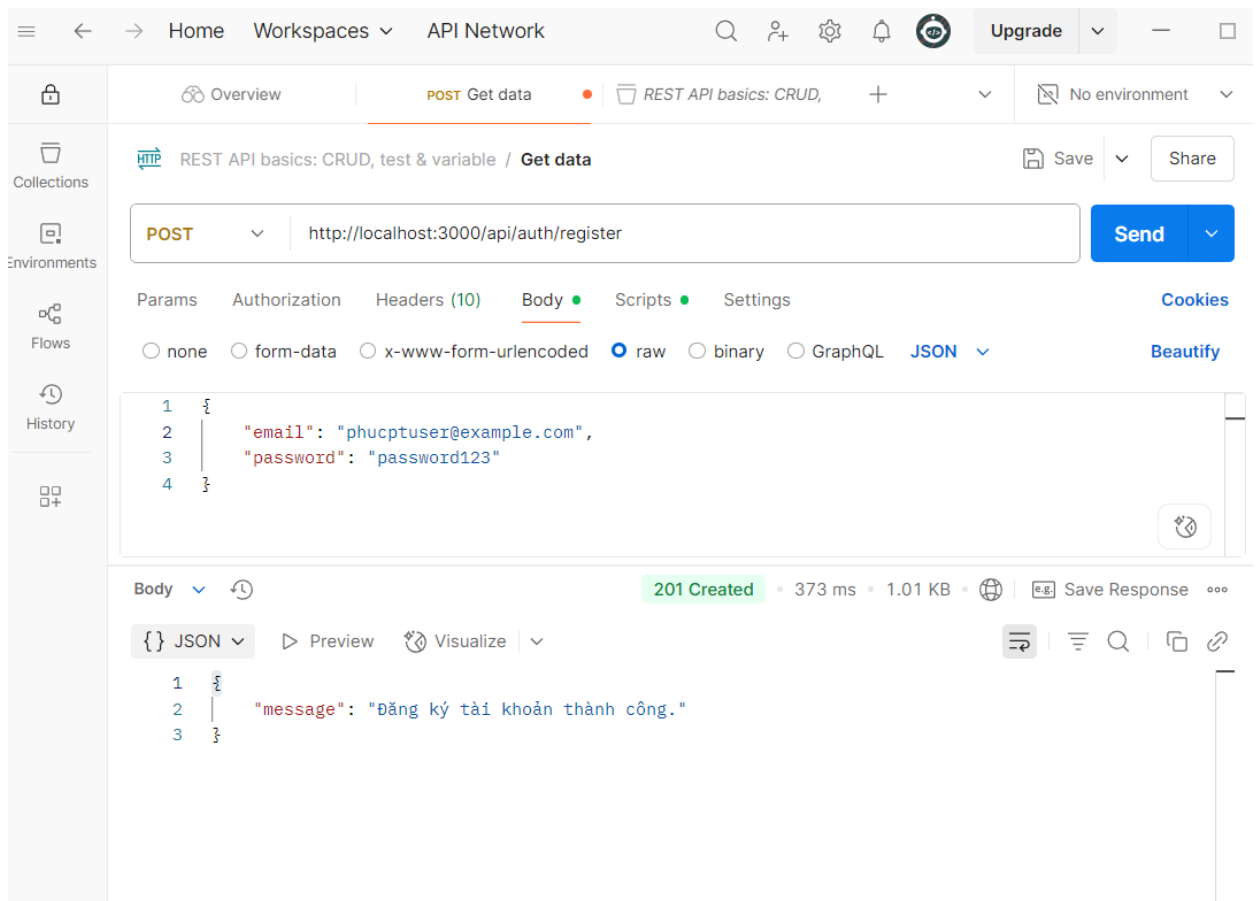
- **Method:** POST
- **URL:** `http://localhost:3000/api/auth/register`
- **Body (JSON):**

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

(Bạn có thể thay đổi email để đăng ký nhiều tài khoản khác nhau, ví dụ: "admin@example.com")

- **Expected Output:**

```
{
  "message": "Đăng ký tài khoản thành công."
}
```



✓ 2. Đăng nhập bằng Passport Local Strategy (Session-based)

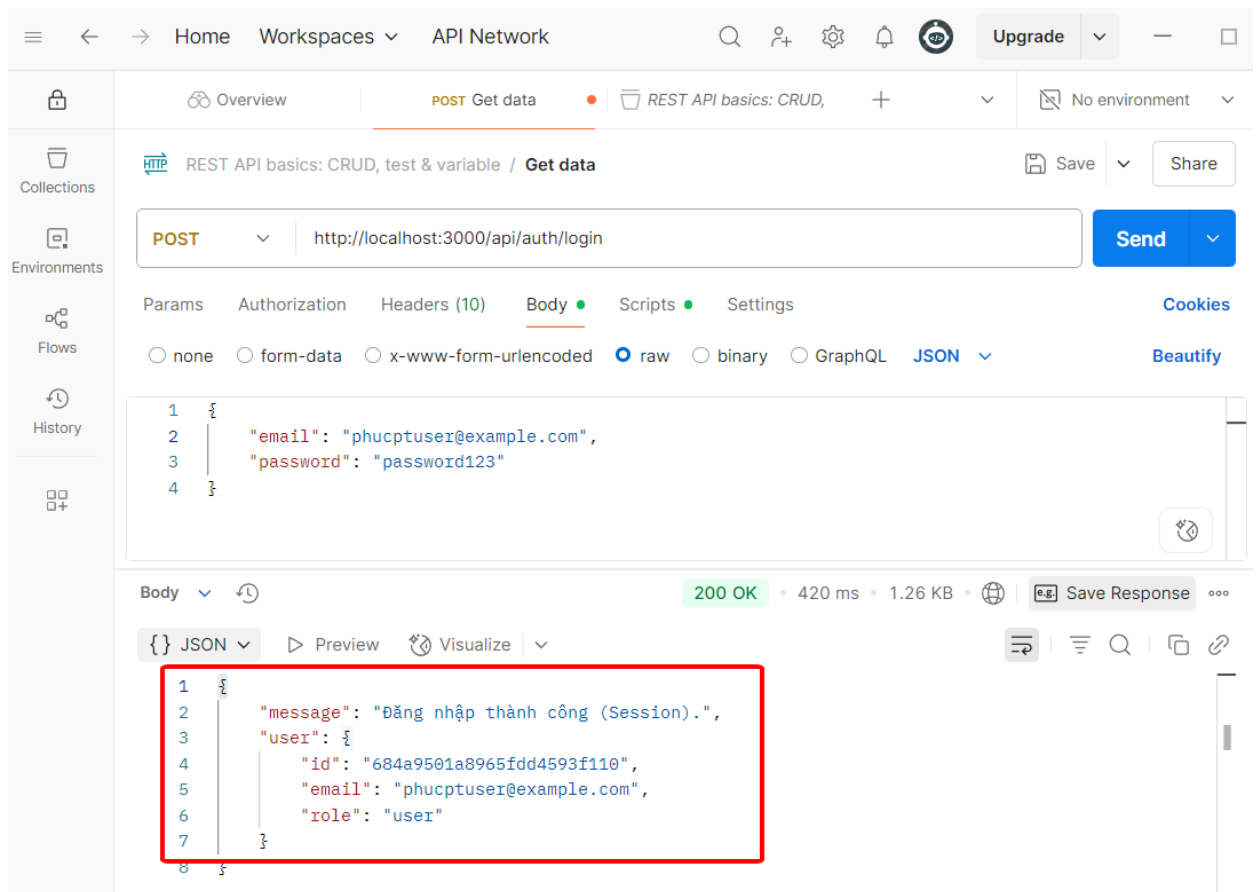
- **Method:** POST
- **URL:** `http://localhost:3000/api/auth/login`
- **Body (JSON):**

```
{  "email": "user@example.com",  "password": "password123"}
```

- **Expected Output:**

```
{  "message": "Đăng nhập thành công (Session).",  "user": {    "id": "...",    "email": "user@example.com",    "role": "user"  }}
```

id sẽ là một ID thực tế từ MongoDB.

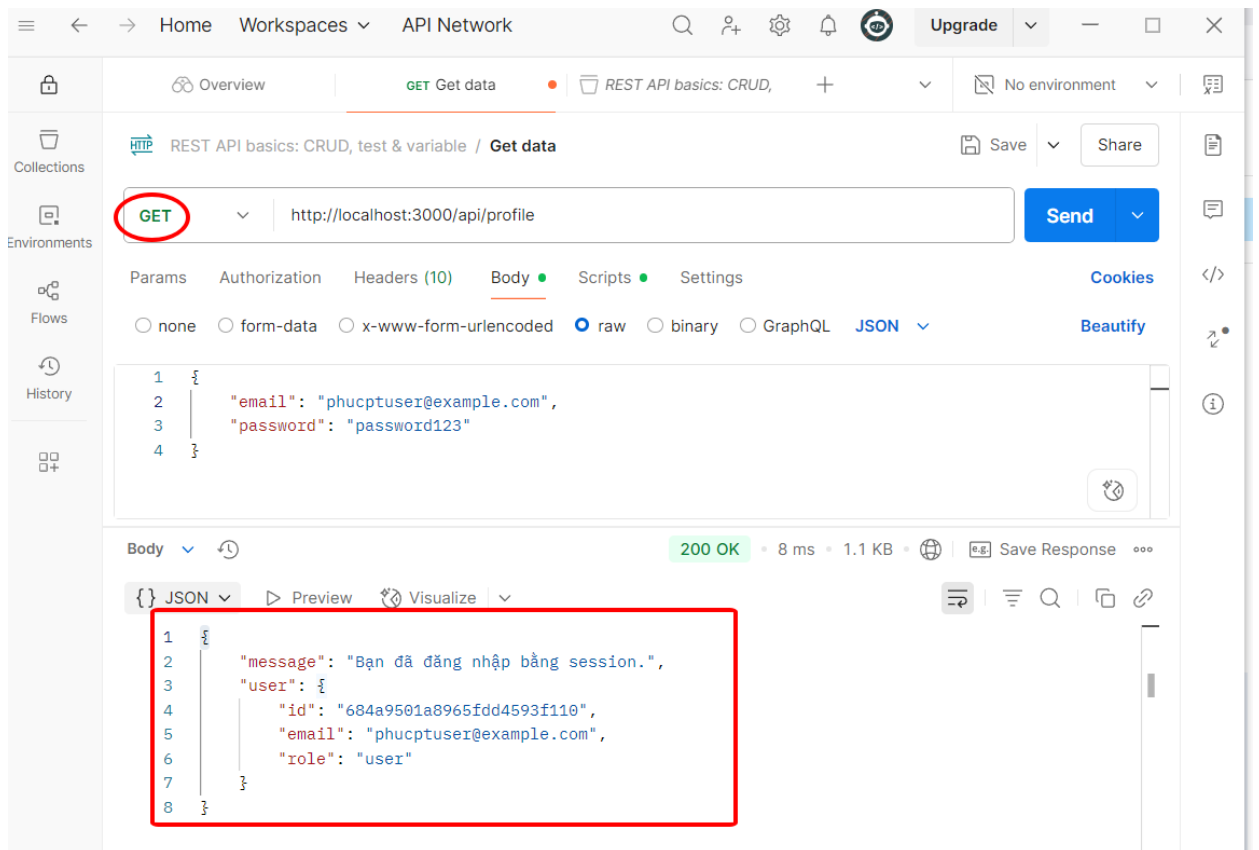


- ⚠️ **Lưu ý:** Cookie session sẽ được tự động lưu bởi Postman/Thunder Client nếu bật tính năng quản lý cookie (thường là mặc định).

✅ 3. Truy cập route `/profile` (chỉ người dùng đã đăng nhập mới vào được - Session-based)

- Điều kiện:** Bạn đã đăng nhập thành công ở bước 2 và Postman/Thunder Client đã lưu cookie session.
- Method:** GET
- URL:** `http://localhost:3000/api/profile`
- Expected Output:**

```
{
  "message": "Bạn đã đăng nhập bằng session.",
  "user": {
    "id": "...",
    "email": "user@example.com",
    "role": "user"
  }
}
```



❌ 3a. Truy cập route `/profile` khi chưa đăng nhập (Session-based)

- **Điều kiện:** Đảm bảo bạn **chưa** đăng nhập (ví dụ: sau khi đã đăng xuất hoặc khởi động lại Postman/Thunder Client).
- **Method:** GET
- **URL:** `http://localhost:3000/api/profile`
- **Expected Output:**

```
{
  "error": "Bạn chưa đăng nhập. Vui lòng đăng nhập để truy cập."
}
```

Expected Status Code: 401 Unauthorized

GET ▼ | http://localhost:3000/api/admin Send ▼

Params Auth Headers (11) Body ● Scripts ● Settings Cookies

Headers 👁 10 hidden

	Key	Value	D...	...	Bulk Edit	Presets ▼
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6I...				
	Key	Value	Description			

Body ▼ 🕒 401 Unauthorized • 5 ms • 1 KB • 🌐 | Save Response ...

{ } JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```

1 {
2   |   "error": "Bạn chưa đăng nhập."
3   }

```

✅ 4. Đăng nhập trả về JWT Token

- **Method:** POST
- **URL:** http://localhost:3000/api/auth/login-jwt
- **Body (JSON):**

```

{
  "email": "user@example.com",
  "password": "password123"
}

```

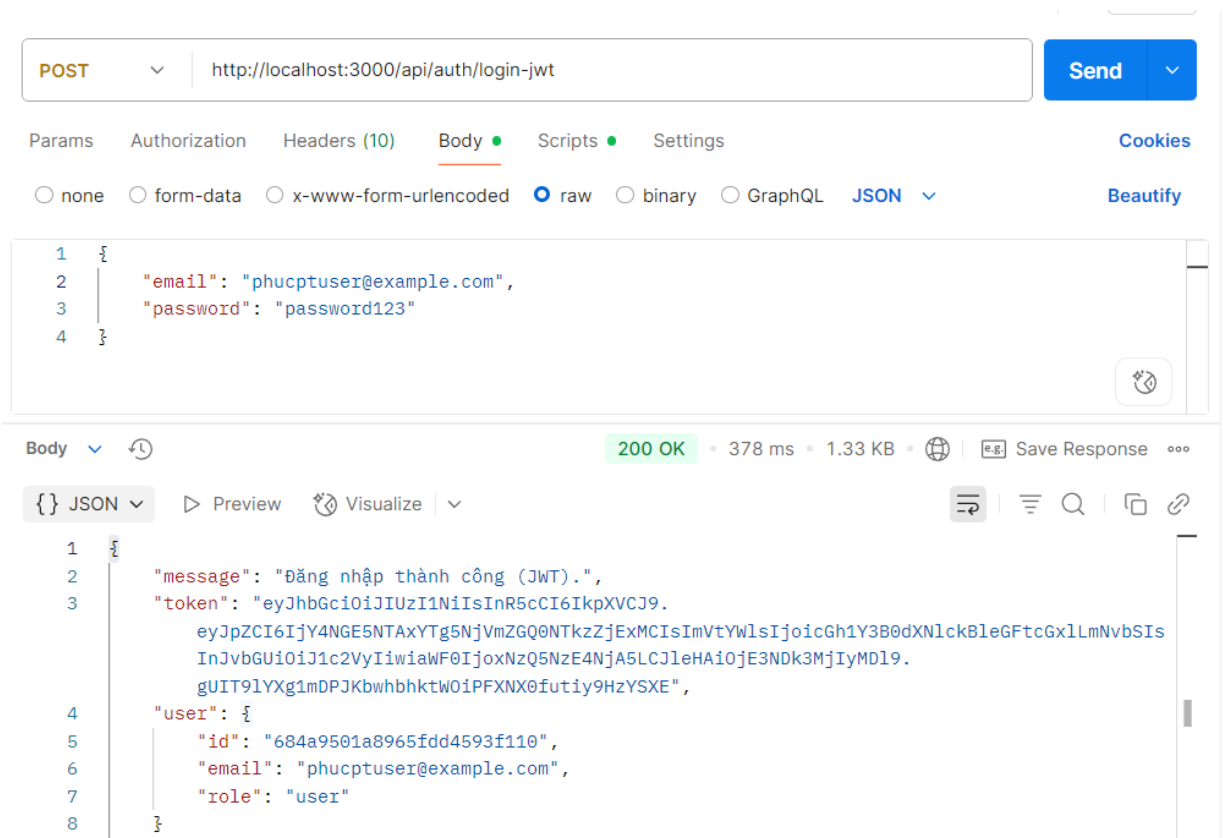
- **Expected Output:**

```

{
  "message": "Đăng nhập thành công (JWT).",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "...",
    "email": "user@example.com",
    "role": "user"
  }
}

```

token và id sẽ là giá trị thực tế.



- ⚠ **Lưu ý:** Sao chép giá trị của token này để sử dụng cho các bước kiểm thử JWT tiếp theo.
 ⇒ eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4NGE5NTAxYTg5NjVmZGQ0NTkzZjExMCIsImVtYWlsIjoicGh1Y3B0dXNlckBleGFtcGxlLmNvbSIsInJvbmUiOiJ1c2VyIiwiaWF0IjoxNzQ5NzE4OTk0LCJleHAiOjE3NDk3MjIyOTR9.NNB6r2wPj4Ex9ivHQNdmZdQUiN6xnjIxWnLHBCMp9fk

✅ 5. Truy cập route /jwt-profile với token

- Điều kiện:** Bạn đã nhận được JWT từ bước 4.
- Method:** GET
- URL:** http://localhost:3000/api/jwt-profile
- Headers:**
 - Key: Authorization
 - Value: Bearer <paste_your_jwt_token_here> (thay thế <paste_your_jwt_token_here> bằng token bạn đã nhận được từ bước 4)
- Expected Output:**

```

{
  "message": "Bạn đã xác thực bằng JWT.",
  "user": {
    "id": "...",
    "email": "user@example.com",
    "role": "user"
  }
}

```

GET ▼ | http://localhost:3000/api/jwt-profile Send ▼

Params Auth Headers (11) Body ● Scripts ● Settings Cookies

Headers 10 hidden

	Key	Value	D...	...	Bulk Edit	Presets ▼
☒	Authorization	<u>Bearer eyJhbGciOiJIUzI1NiIsInR...</u>				
	Key	Value	Description			

Body ▼ ↺ 200 OK • 8 ms • 1.09 KB • 🌐 | e.g. Save Response ...

{} JSON ▼ ▶ Preview 🔍 Visualize ▼ 🔧 ☰ 🔍 📄 🔗

```

1  {
2    "message": "Bạn đã xác thực bằng JWT.",
3    "user": {
4      "id": "684a9501a8965fdd4593f110",
5      "email": "phucptuser@example.com",
6      "role": "user"
7    }
8  }

```

❌ 5a. Truy cập route /jwt-profile với token không hợp lệ/hết hạn

- **Điều kiện:** Sử dụng một token đã hết hạn hoặc một chuỗi token sai.
- **Method:** GET
- **URL:** http://localhost:3000/api/jwt-profile
- **Headers:**
 - Key: Authorization
 - Value: Bearer invalid_or_expired_token
- **Expected Output:**

```

{
  "error": "Token không hợp lệ hoặc đã hết hạn."
}

```

Expected Status Code: 403 Forbidden

GET ▼ http://localhost:3000/api/jwt-profile

Params Auth Headers (12) Body ● Scripts ● Settings

Headers 👁 10 hidden

	Key	Value	D...	...	Bulk E
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6I...			
☐	A				
	Key	Value	Description		

Body ▼ 🔄 401 Unauthorized • 8 ms • 1.03 KB • 🌐 | 📄 Sav

{} JSON ▼ ▶ Preview 🔍 Visualize ▼ 🔄 | ☰

```

1  {
2  |   "error": "Không có token xác thực được cung cấp."
3  }
```

✅ 6. Đăng xuất (Session-based)

- **Điều kiện:** Bạn đã đăng nhập bằng session ở bước 2.
- **Method:** GET
- **URL:** http://localhost:3000/api/auth/logout
- **Expected Output:**

```
{
  "message": "Đăng xuất thành công."
}
```

GET http://localhost:3000/api/auth/logout Send

Params Auth Headers (11) Body Scripts Settings Cookies

Headers 10 hidden

	Key	Value	D...	...	Bulk Edit	Presets
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6I...				
	Key	Value	Description			

Body 200 OK • 15 ms • 1.16 KB • Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "Đăng xuất thành công."
3 }
```

 Bước 6: Test thêm chức năng phân quyền (Admin Roles)

1. Chuẩn bị tài khoản Admin:

- Đăng ký một tài khoản mới (ví dụ: admin@example.com, adminpassword) bằng cách sử dụng Bước 1.
- **Sau khi đăng ký**, sử dụng MongoDB Compass (hoặc công cụ quản lý MongoDB khác) để kết nối vào database authDemo.
- Tìm tài khoản admin@example.com trong collection users.
- Chỉnh sửa trường role của tài khoản này từ "user" thành "admin".

POST http://localhost:3000/api/auth/register Send

Params Auth Headers (11) Body Scripts Settings Cookies

raw JSON Beautify

```

1  {
2    "email": "admin@example.com",
3    "password": "adminpassword"
4  }

```

Body 201 Created • 366 ms • 1.01 KB • Save Response

JSON Preview Visualize

```

1  {
2    "message": "Đăng ký tài khoản thành công."
3  }

```

CONNECTIONS (1)

localhost:27017

- EmpDatabase
- EmpDatabase1
- EmpDatabase_2025
- EmployeeDatabase
- NoteApp
- admin
- authDemo
 - users
 - config
 - demo
 - employee_management
 - local

Documents 2 Aggregations Schema Indexes 1 Validation

Generate query Explain Reset Find

Project { field: 0 }

Sort { field: -1 } or [['field', -1]]

Collation { locale: 'simple' }

Index Hint { field: -1 }

Max Time MS 60000

Skip 0 Limit 0

ADD DATA EXPORT DATA UPDATE DELETE

```

1  _id: ObjectId('684a8f47a8965fdd4593f104')
2  email: "testuser@example.com"
3  role: "user"
4  createdAt: 2025-06-12T08:26:47.545+00:00
5  salt: "ac435e6d6fe6048c06e3a780c8a8bf6c07a16c45976870cd5dd232a944754e8"
6  hash: "f14b1a95625921429f464bfa04390df7c35a9b3d9fa0848e6bf132210ee0072ad40f83..."
7  __v: 0

```

Objec Strin Date Strin Int32

2. Đăng nhập Admin và lấy JWT

- Đăng nhập với tài khoản admin@example.com bằng endpoint `http://localhost:3000/api/auth/login-jwt` (tương tự Bước 4).
- Lưu lại **JWT Token** nhận được cho tài khoản admin này.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/api/auth/login
- Body (JSON):**

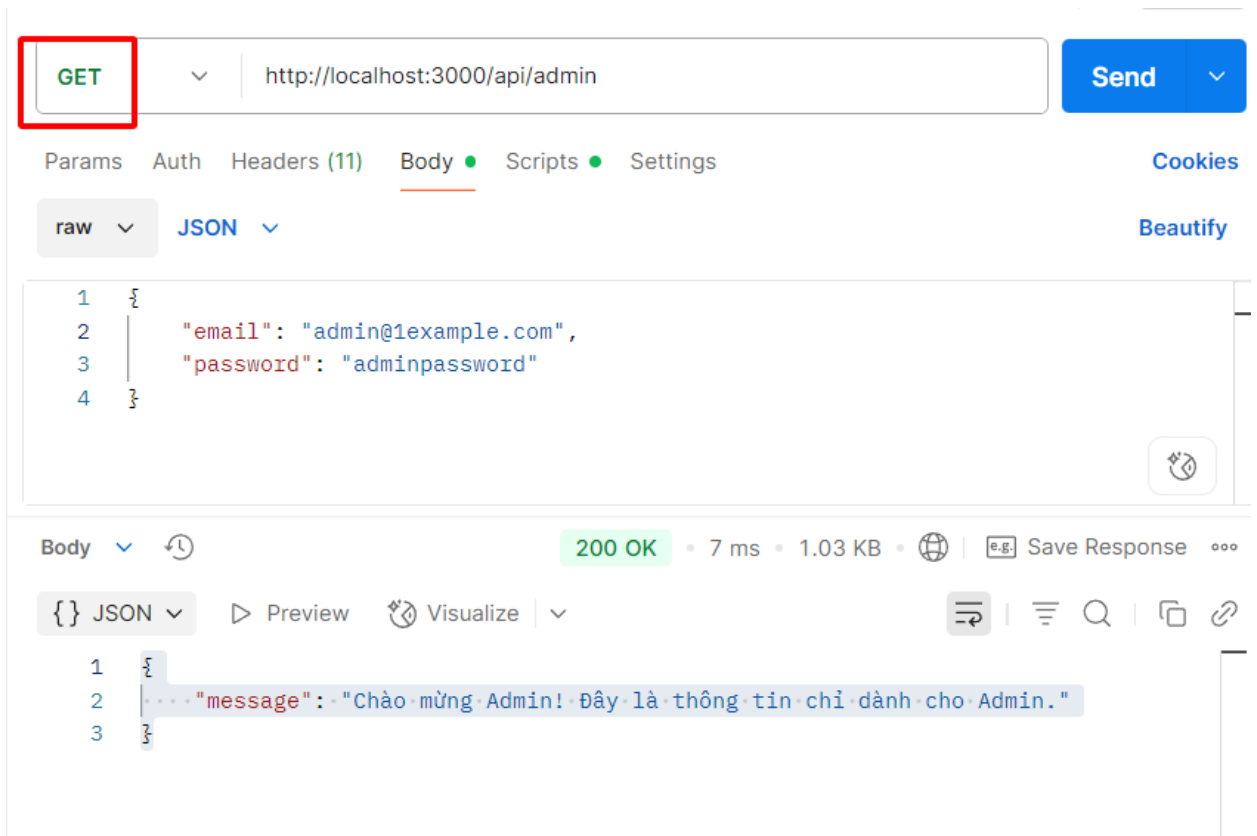
```
{  "email": "admin@1example.com",  "password": "adminpassword"}
```
- Status:** 200 OK
- Response Time:** 379 ms
- Response Size:** 1.25 KB
- Response Body (JSON):**

```
{  "message": "Đăng nhập thành công (Session).",  "user": {    "id": "684a9b49a8965fdd4593f125",    "email": "admin@1example.com",    "role": "admin"  }}
```

✓ 2a. Truy cập route /admin (Session-based, chỉ Admin)

- **Điều kiện:** Đã đăng nhập bằng session với tài khoản **admin** (tương tự Bước 2).
- **Method:** GET
- **URL:** http://localhost:3000/api/admin
- **Expected Output:**

```
{  "message": "Chào mừng Admin! Đây là thông tin chỉ dành cho Admin."}
```



❌ 2b. Truy cập route /admin với tài khoản User (Session-based)

- **Điều kiện:** Đã đăng nhập bằng session với tài khoản **user** (tương tự Bước 2).
- **Method:** GET
- **URL:** `http://localhost:3000/api/admin`
- **Expected Output:**

```
{  "error": "Truy cập bị từ chối: Bạn không có quyền quản trị."}
```

Expected Status Code: 403 Forbidden

✅ 3a. Truy cập route /jwt-admin (JWT-based, chỉ Admin)

- **Điều kiện:** Sử dụng JWT Token của tài khoản **admin** (lấy ở bước chuẩn bị Admin).
- **Method:** GET
- **URL:** `http://localhost:3000/api/jwt-admin`
- **Headers:**
 - Key: Authorization
 - Value: Bearer <paste_your_admin_jwt_token_here>
- **Expected Output:**

```
{  "message": "Chào mừng Admin! Đây là thông tin chỉ dành cho Admin (qua JWT)."}}
```

POST http://localhost:3000/api/auth/login-jwt Send

Params Auth Headers (11) Body Scripts Settings Cookies Beautify

raw JSON

```
1 {
2   "email": "admin@1example.com",
3   "password": "adminpassword"
4 }
```

Body 200 OK • 378 ms • 1.32 KB • Save Response

{ JSON Preview Visualize

```
1 {
2   "message": "Đăng nhập thành công (JWT).",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4NGE5YjQ5YTg5NjVmZGQ0NTkzZjEyNSIsImVtYWlsIjoieiYWRtaW5AMWV4YW1wYUuY29tIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzQ5NzIwODAzLCJleHAiOjE3NDk3MjQ0MDN9.Vr9tcI6nB_yfDQiy5hhGBUXHQhT3mxFsti0j2brfh9g",
4   "user": {
5     "id": "684a9b49a8965fdd4593f125",
6     "email": "admin@1example.com",
7     "role": "admin"
8   }
9 }
```

The image shows a REST client interface with a POST request to `http://localhost:3000/api/auth/login-jwt`. The request body is a JSON object with `email` and `password` fields. The response is a 200 OK status with a response time of 378 ms and a size of 1.32 KB. The response body is a JSON object containing a `message`, a `token`, and a `user` object. Red arrows indicate the flow from the request body to the response body, specifically from the `email` and `password` fields to the `token` and `user` fields.

REST API basics: CRUD, test & variable / **Get data** Save Share

GET ▼ Send ▼

Params Auth **Headers (11)** Body ● Scripts ● Settings Cookies

Headers 10 hidden

	Key	Value	D...	...	Bulk Edit	Presets ▼
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR...				
	Key	Value				Description

Body ▼ ↺ 200 OK • 8 ms • 1.04 KB • 🌐 | Save Response ...

{} JSON ▼ ▶ Preview 🔍 Visualize ▼ 🔧 ☰ 🔍 📄 🔗

```

1  {
2  |   "message": "Chào mừng Admin! Đây là thông tin chỉ dành cho Admin (qua JWT).\"
3  | }

```

❌ 3b. Truy cập route /jwt-admin với tài khoản User (JWT-based)

- **Điều kiện:** Sử dụng JWT Token của tài khoản **user** (lấy ở Bước 4).
- **Method:** GET
- **URL:** `http://localhost:3000/api/jwt-admin`
- **Headers:**
 - Key: Authorization
 - Value: Bearer <paste_your_user_jwt_token_here>
- **Expected Output:**

```

{
  "error": "Truy cập bị từ chối: Không có quyền Admin."
}

```

Expected Status Code: 403 Forbidden

✅ 4. Truy cập route /users (JWT-based, chỉ Admin)

- **Điều kiện:** Sử dụng JWT Token của tài khoản **admin** (lấy ở bước chuẩn bị Admin).
- **Method:** GET
- **URL:** `http://localhost:3000/api/users`

- **Headers:**
 - Key: Authorization
 - Value: Bearer <paste_your_admin_jwt_token_here>
- **Expected Output:**

```
{
  "message": "Danh sách tất cả người dùng.",
  "users": [
    {
      "id": "...",
      "email": "admin@example.com",
      "role": "admin",
      "createdAt": "..."
    },
    {
      "id": "...",
      "email": "user@example.com",
      "role": "user",
      "createdAt": "..."
    }
  ]
}
```

Danh sách sẽ bao gồm tất cả các tài khoản bạn đã đăng ký.

GET

▼

http://localhost:3000/api/users

Send

Params

Auth

Headers (11)

Body

Scripts

Settings

Headers

10 hidden

	Key	Value	D...	...	Bulk Edit	Pre
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR...				
	Key	Value	Description			

Body

▼

🕒

200 OK

• 13 ms • 1.82 KB • 🌐

Save Response

{ } JSON ▼

▶ Preview

🔍 Visualize ▼

🔄

☰

🔍

```

1  {
2      "message": "Danh sách tất cả người dùng.",
3      "users": [
4          {
5              "id": "684a7d41b8a03b602f06a9b1",
6              "email": "user@example.com",
7              "role": "user",
8              "createdAt": "2025-06-12T09:36:31.880Z"
9          },
10         {
11             "id": "684a7d41b8a03b602f06a9b2",
12             "email": "admin@example.com",
13             "role": "admin",
14             "createdAt": "2025-06-12T09:36:31.881Z"
15         },
16         {
17             "id": "684a87626d9dd01ba4fbd983",
18             "email": "usera@example.com",
19             "role": "user"
20         }
21     ]
22 }

```

Checklist hoàn tất

Dự án khởi động thành công	☒
Đăng ký tài khoản thành công	☒
Đăng nhập bằng session thành công	☒
Đăng nhập trả về JWT thành công	☒
Truy cập route cá nhân thành công	☒
Có thể logout thành công	☒
Phân quyền giữa user và admin hoạt động đúng	☒

Tips bổ sung

Dùng Postman collection

Nhóm các request lại để dễ quản lý

Gán biến môi trường trong Postman	Dùng <code>{{baseUrl}}</code> = <code>http://localhost:3000</code>
Tự động parse token	Dùng script trong Postman để lấy token từ response
Kiểm tra cookie session	Trong tab "Cookies" của Postman
Log lỗi đầy đủ	Dùng <code>console.log()</code> hoặc debugger để debug

File Postman mẫu (Postman Collection JSON)

Bạn có thể export collection này trong Postman để tái sử dụng nhiều lần:

json

```
{
  "info": {
    "name": "Auth System APIs",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Register",
      "request": {
        "method": "POST",
        "header": [],
        "body": {
          "mode": "raw",
          "raw": "{\n \"email\": \"user@example.com\", \n \"password\": \"123456\" \n}"
        },
        "url": {
          "raw": "http://localhost:3000/api/auth/register",
          "host": ["localhost"],
          "port": "3000",
          "path": ["api", "auth", "register"]
        }
      },
      "response": []
    },
    {
      "name": "Login with Session",
      "request": {
        "method": "POST",
        "header": [],
        "body": {
          "mode": "raw",
          "raw": "{\n \"email\": \"user@example.com\", \n \"password\": \"123456\" \n}"
        },
        "url": {
          "raw": "http://localhost:3000/api/auth/login",
          "host": ["localhost"],
          "port": "3000",
          "path": ["api", "auth", "login"]
        }
      },
      "response": []
    },
    {
      "name": "Profile (Session)",
      "request": {
        "method": "GET",
        "header": [],

```

```
"url": {  
  "raw": "http://localhost:3000/api/auth/profile",  
  "host": ["localhost"],  
  "port": "3000",  
  "path": ["api", "auth", "profile"]  
}  
}  
},
```

Tổng kết

 Bạn đã hoàn tất quá trình chạy và test toàn bộ chức năng xác thực trong dự án Auth System.

 Đây là bước quan trọng để hiểu rõ cách hoạt động của:

- Xác thực local (Passport + Session)
- Xác thực stateless (JWT)
- Phân quyền giữa user và admin
- Bảo vệ route dựa trên trạng thái đăng nhập

RÈN LUYỆN

Bài tập 1: Authentication System Cơ Bản

Mục tiêu:

Giúp sinh viên nắm vững cách thiết lập xác thực người dùng bằng tài khoản local (email/password), tích hợp Passport, và bảo vệ route.

Chức năng chính:

- Đăng ký tài khoản
- Đăng nhập
- Đăng xuất
- Truy cập route cá nhân sau khi đăng nhập

Công nghệ sử dụng:

- Node.js + Express
- MongoDB (Mongoose)
- Passport.js + passport-local
- express-session
- dotenv

Cấu trúc thư mục gợi ý:

```
auth-basic/  
├── config/  
│   ├── db.js  
│   └── passport.js  
├── models/  
│   └── User.js  
├── routes/  
│   └── authRoutes.js  
├── controllers/  
│   └── authController.js  
├── middleware/  
│   └── authMiddleware.js  
├── .env  
├── app.js  
└── package.json
```

Tóm tắt các bước thực hiện:

1. Khởi tạo project và cài đặt dependencies.
2. Thiết lập kết nối MongoDB.
3. Tạo schema User với passport-local-mongoose.
4. Cấu hình Passport và session.
5. Viết controller xử lý register/login/logout.
6. Bảo vệ route bằng middleware kiểm tra đăng nhập.
7. Test API bằng Postman.

Test case:

1	Đăng ký tài khoản mới	{ "email": " user@example.com ", "password": "123456" }	Trả về success
2	Đăng nhập thành công	{ "email": " user@example.com ", "password": "123456" }	Trả về user info
3	Truy cập route <code>/profile</code> khi chưa đăng nhập	GET <code>/profile</code>	401 Unauthorized
4	Đăng xuất	GET <code>/logout</code>	200 OK

Bài tập 2: Token-Based Authentication System

Mục tiêu:

Giúp sinh viên hiểu và áp dụng xác thực dựa trên token (JWT), xây dựng hệ thống stateless phù hợp cho ứng dụng di động hoặc API backend.

Chức năng chính:

- Đăng nhập trả về JWT
- Xác thực token để truy cập route
- Kiểm tra thời gian sống của token
- Phân quyền đơn giản giữa user/admin

Công nghệ sử dụng:

- Node.js + Express
- MongoDB (Mongoose)
- JSON Web Token (jsonwebtoken)
- Bcryptjs
- dotenv

Cấu trúc thư mục gợi ý:

```
jwt-auth/  
├── models/  
│   └── User.js  
├── routes/  
│   └── authRoutes.js  
├── controllers/  
│   └── authController.js  
├── middleware/  
│   └── jwtMiddleware.js  
├── utils/  
│   └── jwtUtils.js  
├── .env  
├── app.js  
└── package.json
```

Tóm tắt các bước thực hiện:

1. Khởi tạo project và cài đặt dependencies.
2. Thiết lập kết nối MongoDB.
3. Tạo schema User với bcrypt hash password.
4. Viết controller xử lý login và generate token.
5. Bảo vệ route bằng middleware kiểm tra token.
6. Thêm phân quyền đơn giản dựa trên role trong token.
7. Test API bằng Postman.

Test case:

1	Đăng nhập thành công	{ "email": " user@example.com ", "password": "123456" }	Trả về JWT token
2	Gửi token hợp lệ lên /profile	Bearer <token>	Trả về user info
3	Gửi token hết hạn lên /profile	Bearer <old_token>	403 Forbidden
4	Đăng nhập với mật khẩu sai	{ "email": " user@example.com ", "password": "wrongpass" }	401 Unauthorized

Bài tập 3: Multi-Strategy Auth System (Nâng cao)

Mục tiêu:

Xây dựng hệ thống hỗ trợ nhiều phương thức xác thực: local (Passport) và token (JWT), đồng thời triển khai phân quyền chi tiết hơn.

Chức năng chính:

- Đăng ký, đăng nhập bằng email/password
- Đăng nhập bằng JWT
- Phân quyền giữa `user`, `admin`
- Bảo vệ route theo từng vai trò
- Hỗ trợ logout cả session và blacklist token (nếu dùng JWT)

Công nghệ sử dụng:

- Node.js + Express
- MongoDB (Mongoose)
- Passport.js, passport-local, passport-local-mongoose
- JSON Web Token (`jsonwebtoken`)
- `express-session`
- `dotenv`, `morgan`, `helmet`

Cấu trúc thư mục gợi ý:

```
multi-auth/  
├── config/  
│   ├── db.js  
│   └── passport.js  
├── models/  
│   └── User.js  
├── routes/  
│   ├── authRoutes.js  
│   └── protectedRoutes.js  
├── controllers/  
│   ├── authController.js  
│   └── userController.js  
├── middleware/  
│   ├── authMiddleware.js  
│   └── jwtMiddleware.js  
├── utils/  
│   └── jwtUtils.js  
├── .env  
├── app.js  
└── package.json
```

Tóm tắt các bước thực hiện:

1. Khởi tạo project và cài đặt dependencies.
2. Thiết lập kết nối MongoDB.
3. Tạo schema User với `passport-local-mongoose`.
4. Cấu hình Passport + Session và JWT.
5. Viết controller xử lý login bằng cả hai phương thức.
6. Bảo vệ route bằng middleware kiểm tra đăng nhập và vai trò.
7. Thêm chức năng phân quyền giữa `user` và `admin`.
8. Test API bằng Postman.

Test case:

1	Đăng ký tài khoản mới	{ "email": " user@example.com ", "password": "123456" }	Trả về success
---	-----------------------	--	----------------

2	Đăng nhập bằng session	POST /login với email/pass	Trả về user info
3	Đăng nhập bằng JWT	POST /login-jwt với email/pass	Trả về token
4	Truy cập route /admin với user thường	GET /admin	403 Forbidden
5	Truy cập route /admin với admin	GET /admin	Trả về message “Chào mừng Admin!”